

Département d'Informatique

Licence fondamentale : Sciences Mathématiques et Informatique (SMI)

Année universitaire : 2024-2025

Projet de Fin d'Etudes

Intitulé :

L'application de l'IA pour l'analyse et la prédiction des défauts dans les systèmes de distribution électrique

Elaboré par :

- ✓ AHANSAL ZAKARIA
- ✓ TABZAOUI OUSSAMA

Encadré par :

Pr. My Driss EL OUADGHIR

Soutenu le 1 juillet 2025 devant le jury composé de :

M. Hamid Bourray, Professeur - Faculté des Sciences Meknès

M. Moulay Driss El Ouadghiri, Professeur - Faculté des Sciences Meknès

M. Zakaria Lahbi, Professeur - Faculté des Sciences Meknès

Remerciements

C'est avec un grand plaisir que nous réservons ces lignes en signe de gratitude et de reconnaissance à tous ceux qui ont contribué de près ou de loin à l'élaboration de ce travail.

Tout d'abord et avec les mots d'appréciation et de respect, on remercie infiniment notre encadrent Mr. Moulay Driss EL oudghiri pour son encadrement et sa disponibilité malgré ses obligations professionnelles, ainsi que pour ses conseils judicieux, précieux et ses directives pertinentes pour l'intérêt qu'il a porté à notre sujet.

Notre grand respect et nos sincères remerciements vont à Mr. Khtou Otmame qui ont accepté de nous superviser et nous guider avec leurs conseils précieux lors de la réalisation de ce projet.

On exprime aussi nos sincères gratitudes à tous les enseignants de la Faculté des Sciences Meknès pour la qualité de la formation qu'ils nous ont dispensée.

Enfin, nous tenons à remercier tous les membres du jury pour nous avoir accordé l'honneur de juger notre modeste travail.

Résumé

À l'heure où la transition énergétique et la digitalisation des infrastructures deviennent des priorités, l'intelligence artificielle s'impose comme un levier incontournable pour moderniser les réseaux électriques. Ce projet s'inscrit dans une démarche innovante visant à renforcer la résilience et l'intelligence des systèmes de distribution. Ce projet de fin d'études s'inscrit dans le cadre de l'intégration de l'intelligence artificielle (AI) dans les systèmes de distribution électrique, dans le but de détecter et de prédire les défauts. Face à la complexité croissante des réseaux électriques, les méthodes traditionnelles de maintenance se révèlent souvent insuffisantes. Ce travail propose une approche basée sur l'apprentissage automatique, notamment les réseaux de neurones et les arbres de décision, pour analyser les données historiques et identifier les signes avant-coureurs de pannes. Les étapes principales incluent la collecte, le traitement des données, l'entraînement des modèles et l'évaluation de leurs performances. Les résultats obtenus montrent que les modèles d'IA offrent une meilleure précision dans la détection précoce des anomalies, ce qui permet d'optimiser les interventions de maintenance, d'améliorer la fiabilité du réseau et de réduire les temps d'arrêt.

Abstract

As the energy transition and infrastructure digitalization become top priorities, artificial intelligence (AI) is emerging as a key driver for modernizing electrical networks. This project is part of an innovative approach aimed at enhancing the resilience and intelligence of power distribution systems. It focuses on the integration of AI into electrical distribution networks to detect and predict faults. Given the growing complexity of these systems, traditional maintenance methods often prove inadequate. This work proposes a machine learning-based approach—specifically using neural networks and decision trees—to analyze historical data and identify early signs of failures. The main stages include data collection, preprocessing, model training, and performance evaluation. The results show that AI models offer greater accuracy in early anomaly detection, enabling more efficient maintenance, improved network reliability, and reduced downtime.

Sommaire

Remerciements	2
Résumé	3
Abstract	4
1 Introduction à l'intelligence artificielle	12
1.1 Définition de l'intelligence artificielle	12
1.2 Le Machine Learning (apprentissage automatique)	12
1.3 Le Deep Learning (apprentissage profond)	13
2 Les outils et technologies utilisés	14
2.1 Environnements de développement	14
2.1.1 Python	14
2.1.2 Jupyter Notebook	14
2.1.3 Google Colab	14
2.2 Bibliothèques IA : Scikit-learn, TensorFlow, Keras	14
2.2.1 Scikit-learn	14
2.2.2 TensorFlow	15
2.2.3 Keras	15
2.2.4 XGBoost (eXtreme Gradient Boosting)	15
2.3 Outils de visualisation et traitement de données : Pandas, Matplotlib :	16
2.3.1 Pandas	16
2.3.2 Numpy	16
2.3.3 Matplotlib	17
2.3.4 Seaborn	17
2.3.5 imbalanced-learn	17
2.3.6 Pydot	18
2.3.7 Graphviz	18
2.3.8 Squarify	18
2.3.9 umap-learn	19
2.4 Outils de gestion de version et stockage : Git, GitHub, GitLab	19

2.5	Outils de présentation (Website) : Streamlit	20
3	Présentation du Dataset	22
3.1	Constitution du dataset pour l'entraînement des modèles	22
3.2	Contenu du dataset	22
3.3	Prétraitement des données	23
3.4	Analyse exploratoire des données	24
3.5	Electrical fault Dataset Analysis Techniques	25
3.5.1	distribution des fonctionnalités d'entrée importées	25
3.5.2	Carte thermique de corrélation	26
3.5.3	Répartition par type de défaut	27
3.5.4	Visualisation PCA avec types de défauts	28
3.5.5	Visualisation des données avec t-SNE	29
3.5.6	Visualisation de l'analyse de phase	30
3.5.6.1	Tracé de coordonnées parallèles	30
3.5.7	Analyse de clustering et de silhouette	31
3.5.8	Visualisation PCA 3D	32
3.5.9	Diagramme polaire du courant et des tensions triphasés	33
3.5.10	Importance des fonctionnalités à l'aide de la forêt aléatoire	34
3.5.11	Analyse des composantes symétriques (critique pour les défauts électriques)	35
3.5.12	répartition des types de pannes	36
3.5.13	clustering non supervisé vs types de défauts réels	37
3.5.14	Représentation vectorielle d'un système triphasé	38
3.5.15	visualisation de détection d'anomalies	39
3.5.16	Conclusion	41
4	Détection des défauts électriques à l'aide de l'IA	42
4.1	Application du Machine Learning	42
4.1.1	K-Nearest Neighbors (KNN)	42
4.1.2	Support Vector Machine (SVM)	43
4.1.2.1	Linear :	43
4.1.2.2	RBF :	44
4.1.2.3	Optimized :	44
4.1.3	Random Forest	45
4.1.4	Arbre de décision (Decision Tree)	46
4.1.5	Régression logistique	47
4.1.6	Analyse du modèle Naive Bayes (GaussianNB)	48
4.1.7	SGD Classifier	49
4.1.8	XGBoost	50
4.1.8.1	Comparaison des modèles :	51

4.1.9	Unsprivised Learning :	52
4.1.10	Conclusion sur les modèles supervisés et non supervisés :	52
4.2	Application du Deep Learning	53
4.2.1	CNN training	53
4.2.2	CNN evaluation	53
4.2.3	AdvancedCNN evaluation	54
4.2.4	AdvancedCNN training	55
4.2.5	FNN evaluation	56
4.2.6	FNN training	57
4.2.7	GRU evaluation	58
4.2.8	GRU training	59
4.2.9	Improved FNN evaluation	60
4.2.10	Improved FNN training	61
4.2.11	LSTM evaluation	62
4.2.12	LSTM training	63
4.3	Comparaison des performances des modèles Deep Learning	64
4.4	Tableau de comparaison des modelés Deep Learning	66
4.5	Conclusion sur la comparaison des modèles Deep Learning	66
5	Comparaison des performances des modèles	67
5.1	Métriques utilisées pour l'évaluation	67
5.2	Résultats de la comparaison des modèles	68
5.3	L'importance du volume de données pour l'efficacité du Deep Learning	68
5.4	Interprétation des résultats	69
5.5	Choix du meilleur modèle	69
5.6	Discussion sur les limites des modèles	69
6	Conclusion Générale	70
7	Perspectives Futures	71
8	References bibliographies	72

Listes des Figures

Figure 1 : Distribution des caractéristiques d'entrée (courants et tensions)	18
Figure 2 : Carte thermique de corrélation entre les variables électriques	19
Figure 3 : Répartition des types de défauts dans le dataset	20
Figure 4 : Projection PCA (2D) des types de défauts	21
Figure 5 : Visualisation des données avec t-SNE	22
Figure 6 : Scatter plot des phases de courant et de tension (I_a vs I_b , V_a vs V_b)	22
Figure 7 : Graphique en coordonnées parallèles selon les types de défauts	23
Figure 8 : Score de silhouette selon le nombre de clusters (KMeans)	24
Figure 9 : Visualisation PCA en 3D des défauts électriques	25
Figure 10 : Diagramme polaire des tensions et courants triphasés	26
Figure 11 : Boxplots des composantes symétriques selon les défauts	27
Figure 12 : Analyse des composantes symétriques (I_0 , I_1 , I_2 / V_0 , V_1 , V_2)	28
Figure 13 : Histogramme + diagramme circulaire des défauts simulés	29
Figure 14 : Matrice de confusion pour le clustering non supervisé	30
Figure 15 : Représentation vectorielle des défauts triphasés	31
Figure 16 : Détection d'anomalies avec Isolation Forest	32
Figure 17 : Matrice de confusion du modèle Naive Bayes	39
Figure 18 : Courbe d'apprentissage du modèle SGD Classifier	40
Figure 19 : Matrice de confusion du modèle SGD Classifier	40
Figure 20 : Matrice de confusion du modèle XGBoost	41
Figure 21 : Courbe d'apprentissage du modèle XGBoost	41
Figure 22 : Diagramme radar comparatif des modèles ML	42
Figure 23 : Tableau comparatif des performances des modèles ML	43
Figure 24 : Matrice de confusion du modèle AdvancedCNN	44
Figure 25 : Score F1 par classe pour le modèle AdvancedCNN	45
Figure 26 : Visualisation des erreurs de classification LLG vs LLL – AdvancedCNN	45
Figure 27 : Courbe d'évolution de la précision – AdvancedCNN	46
Figure 28 : Résultats d'entraînement du modèle AdvancedCNN	46
Figure 29 : Résultats de validation du modèle AdvancedCNN	47
Figure 30 : Matrice de confusion du modèle FNN	48
Figure 31 : Visualisation des erreurs du modèle FNN	48

Figure 32 : Courbe d'évolution de la précision – FNN	49
Figure 33 : Répartition des prédictions FNN par classe	49
Figure 34 : Courbe de perte du modèle FNN	50
Figure 35 : Résumé de l'architecture et des hyperparamètres du FNN	50
Figure 36 : Courbe de précision de validation du modèle GRU	51
Figure 37 : Matrice de confusion du modèle GRU	51
Figure 38 : F1-score par classe – modèle GRU	52
Figure 39 : Évolution du loss (perte) – modèle GRU	52
Figure 40 : Précision d'entraînement du modèle GRU	53
Figure 41 : Précision de validation du modèle GRU	53
Figure 42 : Matrice de confusion du modèle ImprovedFNN	54
Figure 43 : Erreurs de classification du modèle ImprovedFNN	54
Figure 44 : Score F1 par classe – modèle ImprovedFNN	55
Figure 45 : Évolution de la précision – ImprovedFNN	55
Figure 46 : Résultats de validation du modèle ImprovedFNN	56
Figure 47 : Courbe d'apprentissage du modèle ImprovedFNN	56
Figure 48 : Architecture et paramètres du modèle ImprovedFNN	57
Figure 49 : Matrice de confusion du modèle LSTM	58
Figure 50 : Confusions entre classes LLL et LLG – modèle LSTM	58
Figure 51 : F1-score par classe – modèle LSTM	59
Figure 52 : Courbe de perte (loss) du modèle LSTM	59
Figure 53 : Précision d'entraînement du modèle LSTM	60
Figure 54 : Précision de validation du modèle LSTM	60
Figure 56 : Signal de courant mesuré en cas de défaut L-L.....	61
Figure 57 : Courbe de fréquence de défauts détectés par type.....	62
Figure 58 : Architecture du réseau GRU avec couches denses.....	63
Figure 59 : Évolution de la courbe de précision pour réseau GRU.....	63
Figure 60 : Répartition du dataset entre entraînement et test.....	64
Figure 61 : Prédictions GRU sur échantillons de validation.....	64
Figure 62 : Densité de probabilité des prédictions GRU.....	65
Figure 63 : Comparaison des scores F1 entre GRU et FNN.....	66

Introduction

Dans un monde de plus en plus dépendant de l'énergie électrique, la fiabilité des réseaux de distribution est devenue un enjeu stratégique majeur. Les interruptions de service, même brèves, peuvent engendrer des conséquences considérables tant sur le plan économique que social. Face à cette réalité, la prévention et la détection précoce des défauts dans les systèmes de distribution électrique représentent un défi technique de premier ordre pour les opérateurs et gestionnaires de réseaux.

Dans un monde de plus en plus dépendant de l'énergie électrique, la fiabilité des réseaux de distribution est devenue un enjeu stratégique majeur. Les interruptions de service, même brèves, peuvent engendrer des conséquences considérables tant sur le plan économique que social. Face à cette réalité, la prévention et la détection précoce des défauts dans les systèmes de distribution électrique représentent un défi technique de premier ordre pour les opérateurs et gestionnaires de réseaux.

Les systèmes de distribution électrique contemporains sont d'une complexité croissante, intégrant des sources d'énergie traditionnelles et renouvelables, des infrastructures vieillissantes et des équipements modernes, le tout devant répondre à des demandes fluctuantes dans un contexte de transition énergétique. Cette complexité multiplie les points de vulnérabilité potentiels et rend les approches conventionnelles de maintenance et de surveillance de moins en moins efficaces.

C'est dans ce contexte que l'intelligence artificielle (IA) émerge comme une solution prometteuse. Les récentes avancées dans le domaine de l'apprentissage automatique, de l'analyse de données massives et des systèmes prédictifs offrent des perspectives inédites pour transformer radicalement la gestion des réseaux électriques. En permettant l'analyse en temps réel de volumes considérables de données hétérogènes, l'IA ouvre la voie à une nouvelle génération de systèmes capables non seulement de détecter les anomalies existantes, mais également de prédire les défaillances avant leur manifestation.

Ce projet se propose d'explorer et de développer des solutions innovantes basées sur l'intelligence artificielle pour répondre aux défis de l'analyse et de la prédiction des défauts

Problématique

Les systèmes de distribution électrique modernes sont de plus en plus complexes, interconnectés et sensibles aux défauts qui peuvent provoquer des interruptions importantes du service, des dommages matériels, voire des risques pour la sécurité publique. En particulier, les lignes de transmission, qui assurent le transport de l'énergie entre les zones de production et les réseaux de distribution, sont soumises à des perturbations telles que les courts-circuits monophasés, biphasés ou triphasés, les défauts à la terre, ou encore les défauts symétriques. Ces événements peuvent entraîner des conséquences graves si elles ne sont pas détectées et traitées rapidement (Glover et al., [1]).

Les méthodes classiques de détection des défauts, basées sur des seuils ou des relais électromécaniques, montrent leurs limites face à l'évolution du réseau électrique : elles manquent souvent de précision, sont lentes à réagir, ou nécessitent des réglages manuels complexes (Hewitson et al., [2]). Aujourd'hui, grâce à l'essor de l'intelligence artificielle, de nouvelles approches basées sur l'apprentissage automatique permettent d'analyser les signaux électriques en temps réel, d'identifier les modèles caractéristiques de défauts et d'anticiper les pannes avant qu'elles ne se produisent (James et al., [3]; Haykin, [4]).

Ainsi, l'enjeu principal de ce projet est d'étudier comment les algorithmes d'IA, en particulier les réseaux de neurones artificiels (ANN), peuvent être appliqués efficacement à l'analyse des données mesurées dans un système de transmission simulé (tensions et courants des trois phases), afin de détecter, classifier et prédire les défauts électriques de manière rapide, fiable et automatique (Zhang et al., [5]; Abur et al., [6]).

Problématique principale :

Comment concevoir un système intelligent capable de détecter et classifier automatiquement les défauts dans un réseau de distribution électrique à partir des mesures de courants et tensions, en utilisant des algorithmes d'intelligence artificielle, tout en garantissant une robustesse, une précision et une rapidité de traitement adaptées aux exigences réelles des réseaux électriques ?

1 Introduction à l'intelligence artificielle

1.1 Définition de l'intelligence artificielle

L'intelligence artificielle (IA) est un domaine de l'informatique qui vise à créer des systèmes capables de simuler l'intelligence humaine. Ces systèmes peuvent accomplir des tâches comme la reconnaissance de la parole, la prise de décision, la traduction automatique, ou encore la détection d'objets dans des images. L'IA repose sur des algorithmes capables d'apprendre à partir de données, de s'adapter à de nouvelles situations et de prendre des décisions de manière autonome.

L'objectif principal de l'IA est de reproduire ou d'imiter certaines capacités cognitives humaines telles que l'apprentissage, la résolution de problèmes, et la compréhension du langage naturel. Aujourd'hui, l'IA est présente dans plusieurs secteurs : santé, finance, transport, industrie, énergie, etc.

1.2 Le Machine Learning (apprentissage automatique)

Le Machine Learning (ML) est une sous-branche de l'intelligence artificielle. Il consiste à développer des algorithmes capables d'apprendre automatiquement à partir de données, sans être explicitement programmés pour chaque tâche. On distingue trois grandes catégories de ML :

L'apprentissage supervisé : L'algorithme apprend à partir d'un ensemble de données étiquetées, c'est-à-dire que chaque exemple d'apprentissage est associé à une sortie connue (ex : type de défaut dans un réseau électrique).

L'apprentissage non supervisé : L'algorithme essaye de détecter des structures ou des regroupements dans les données sans avoir d'étiquette (ex : regroupement de comportements anormaux).

L'apprentissage par renforcement : L'agent apprend en interagissant avec un environnement, en recevant des récompenses ou des pénalités selon ses actions.

Le Machine Learning est largement utilisé pour des tâches de classification, de régression, de détection d'anomalies, ou de prédiction.

1.3 Le Deep Learning (apprentissage profond)

Le Deep Learning (DL) est un sous-domaine du Machine Learning. Il se base sur l'utilisation de réseaux de neurones artificiels comportant plusieurs couches cachées, d'où le terme "profond". Ces réseaux sont capables d'extraire automatiquement des caractéristiques complexes à partir de grandes quantités de données. (Goodfellow et al. [7])

Le Deep Learning est particulièrement efficace dans les domaines où les données sont volumineuses et complexes : reconnaissance d'images, traitement du langage naturel, détection de fraudes, systèmes de recommandation, etc.

Contrairement à certains algorithmes classiques de ML, les modèles de DL nécessitent beaucoup de données et une puissance de calcul élevée, mais offrent souvent une meilleure performance, surtout pour les données non linéaires ou non structurées.

2 Les outils et technologies utilisés

2.1 Environnements de développement

L'environnement de développement joue un rôle crucial dans la mise en œuvre des projets d'intelligence artificielle. Les principaux outils utilisés dans ce projet sont les suivants :

2.1.1 Python

Python est le langage de programmation principal utilisé dans ce projet. Il est largement utilisé dans le domaine de l'intelligence artificielle en raison de sa simplicité, de sa flexibilité et de la richesse de ses bibliothèques dédiées à l'IA et au Machine Learning.

Python offre une syntaxe claire et un large écosystème de bibliothèques pour la manipulation des données, l'entraînement des modèles, et la visualisation des résultats

2.1.2 Jupyter Notebook

Jupyter Notebook est un environnement interactif qui permet de combiner code, texte et visualisations dans un même document. Il est particulièrement adapté pour la création de prototypes et la documentation des étapes du projet. Jupyter est utilisé pour écrire et tester le code Python, et permet une visualisation facile des résultats à chaque étape du processus.

2.1.3 Google Colab

Google Colab est un environnement basé sur Jupyter Notebook qui offre des ressources de calcul gratuites, y compris l'accès à des GPU pour l'entraînement des modèles de Deep Learning. Colab permet d'exécuter des projets d'intelligence artificielle sans avoir besoin de matériel puissant localement, et il facilite le partage des travaux avec d'autres collaborateurs.

2.2 Bibliothèques IA : Scikit-learn, TensorFlow, Keras

Les bibliothèques sont des outils essentiels pour l'implémentation des modèles d'intelligence artificielle. Voici les principales bibliothèques utilisées dans ce projet :

2.2.1 Scikit-learn

Scikit-learn est l'une des bibliothèques les plus populaires pour le Machine Learning en Python. Elle fournit une large gamme d'algorithmes de classification, régression, clustering, et réduction

de dimensionnalité. Dans ce projet, Scikit-learn a été utilisé pour implémenter des modèles de Machine Learning classiques, tels que KNN, SVM, et Random Forest.

Fonctionnalités clés : Prétraitement des données, sélection des modèles, validation croisée, évaluation des performances, etc.

Avantages : Facile à utiliser, très bien documenté, et avec une communauté active.

2.2.2 TensorFlow

TensorFlow est une bibliothèque open-source développée par Google pour le Deep Learning. Elle permet de créer, entraîner et déployer des réseaux de neurones profonds. TensorFlow offre des outils puissants pour travailler avec des modèles de Deep Learning complexes, et sa capacité à s'exécuter sur des GPUs permet un traitement rapide des grandes quantités de données.

Fonctionnalités clés : Gestion des modèles de Deep Learning, déploiement sur des serveurs ou mobiles, calcul distribué, etc.

Avantages : Très puissant, flexible, et adapté aux systèmes de production à grande échelle.

2.2.3 Keras

Keras est une bibliothèque haut-niveau pour le Deep Learning, qui peut fonctionner au-dessus de TensorFlow. Elle simplifie la création de modèles de réseaux de neurones grâce à une interface intuitive et des fonctionnalités prêtes à l'emploi. Dans ce projet, Keras a été utilisé pour implémenter les réseaux de neurones profonds (MLP, CNN).

Fonctionnalités clés : Création de modèles de réseaux de neurones, entraînement et évaluation des modèles, gestion des données.

Avantages : Facile à utiliser et à comprendre, accélère le prototypage des modèles.

2.2.4 XGBoost (eXtreme Gradient Boosting)

XGBoost est une bibliothèque optimisée de boosting par gradient, particulièrement efficace pour les tâches de classification et de régression. Elle est largement utilisée en machine learning pour sa rapidité, sa précision et sa capacité à gérer des données hétérogènes. Dans ce projet, XGBoost a été utilisé pour entraîner un modèle de classification permettant de détecter des défauts à partir des données issues du système de distribution électrique.

Fonctionnalités clés : Implémentation efficace du gradient boosting, gestion automatique des valeurs manquantes, régularisation L1 et L2, parallélisation des calculs.

Avantages : Haute performance sur des données structurés, robustesse face à l'overfitting, temps d'entraînement réduit.

2.3 Outils de visualisation et traitement de données : Pandas, Matplotlib :

La visualisation et le traitement des données sont des étapes importantes avant l'entraînement des modèles d'intelligence artificielle. Les outils suivants ont été utilisés pour cette tâche :

2.3.1 Pandas

Pandas est une bibliothèque Python utilisée pour la manipulation et l'analyse des données. Elle fournit des structures de données puissantes, comme les DataFrames, qui permettent de charger, nettoyer, transformer et analyser les données de manière efficace. Pandas a été utilisé pour gérer et prétraiter les données avant de les passer aux modèles d'apprentissage automatique.

Fonctionnalités clés : Manipulation de données en tables (DataFrames), gestion des valeurs manquantes, agrégation des données.

Avantages : Très efficace pour le traitement des grandes quantités de données, interface intuitive.

2.3.2 Numpy

NumPy (Numerical Python) est une bibliothèque fondamentale pour le calcul scientifique en Python. Elle fournit des structures de données performantes, notamment les tableaux multidimensionnels (ndarray), ainsi qu'un grand ensemble de fonctions pour effectuer des opérations mathématiques et statistiques sur ces tableaux.

Dans le cadre de notre projet, NumPy a été utilisé pour :

- Stocker et manipuler efficacement des données numériques issues des systèmes de distribution électrique.
- Réaliser des transformations de données, telles que la normalisation ou le centrage des valeurs.
- Effectuer des calculs statistiques de base (moyennes, écarts-types, sommes) nécessaires à la préparation des données avant l'entraînement des modèles.
- Optimiser les performances de traitement, en évitant les boucles classiques grâce à des opérations vectorielles rapides.

2.3.3 Matplotlib

Matplotlib est une bibliothèque de visualisation pour Python. Elle permet de créer des graphiques statiques, animés et interactifs. Dans ce projet, elle a été utilisée pour visualiser les performances des modèles, les courbes d'apprentissage, les matrices de confusion, et d'autres graphiques utiles pour analyser les résultats.

Fonctionnalités clés : Création de graphiques en 2D, visualisation des données de façon interactive, intégration avec Pandas pour des visualisations avancées.

Avantages : Très flexible et extensible, permet de créer des visualisations sur mesure.

2.3.4 Seaborn

Seaborn est une bibliothèque Python basée sur Matplotlib, mais elle offre des fonctionnalités avancées pour créer des graphiques statistiques complexes. Seaborn a été utilisé dans ce projet pour visualiser des relations entre différentes variables et créer des visualisations claires des résultats des modèles.

Fonctionnalités clés : Visualisation des distributions de données, relations entre variables, création de heatmaps et de matrices de confusion.

Avantages : Plus simple à utiliser que Matplotlib pour des visualisations complexes, esthétiquement plaisant.

2.3.5 imbalanced-learn

Imbalanced-learn est une bibliothèque Python dédiée au traitement des données déséquilibrés, un problème fréquent en apprentissage automatique, notamment dans les cas de détection de défauts rares. Elle a été utilisée dans ce projet pour appliquer des techniques de rééchantillonnage (oversampling et undersampling) afin d'améliorer la performance des modèles de classification.

Fonctionnalités clés : Méthodes de suréchantillonnage (comme SMOTE), sous-échantillonnage, combinaisons de techniques, intégration avec scikit-learn.

Avantages : Facile à intégrer dans un pipeline de machine learning, améliore la robustesse des modèles face aux classes minoritaires, indispensable pour les tâches de détection d'anomalies ou de défauts.

2.3.6 Pydot

Pydot est une interface Python pour Graphviz, permettant la création, la manipulation et la visualisation de graphes orientés. Dans ce projet, Pydot a été utilisé pour visualiser la structure des modèles, notamment les arbres de décision ou les architectures de réseaux de neurones, facilitant ainsi l'analyse et l'interprétation du flux de traitement.

Fonctionnalités clés : Génération de graphes à partir de descriptions en langage DOT, exportation en formats visuels (PNG, PDF), intégration avec Keras et scikit-learn pour visualiser les modèles.

Avantages : Utile pour documenter visuellement les modèles, facilite la compréhension des flux de données et des décisions prises par les algorithmes, compatible avec divers outils de visualisation.

2.3.7 Graphviz

Graphviz est un outil open source de visualisation de graphes basé sur le langage DOT. Il permet de représenter visuellement des structures complexes comme les arbres, les réseaux ou les dépendances entre éléments. Dans ce projet, Graphviz a été utilisé conjointement avec Pydot pour générer des représentations graphiques des modèles d'apprentissage automatique, en particulier les arbres de décision.

Fonctionnalités clés : Visualisation de graphes orientés ou non orientés, prise en charge du langage DOT, exportation en plusieurs formats graphiques (PDF, PNG, SVG).

Avantages : Visualisation claire et personnalisable des structures de données, outil puissant pour le debugging et la documentation des modèles, largement compatible avec des bibliothèques Python comme Pydot et scikit-learn.

2.3.8 Squarify

Squarify est une bibliothèque Python utilisée pour générer des treemaps, un type de visualisation hiérarchique représentant des données sous forme de rectangles proportionnels. Dans ce projet, Squarify a été utilisé pour visualiser la répartition des variables ou des classes dans des jeux de données déséquilibrés, facilitant ainsi l'analyse exploratoire.

Fonctionnalités clés : Création de treemaps, mise à l'échelle automatique des zones, intégration avec Matplotlib pour personnalisation et export.

Avantages : Permet de visualiser efficacement la hiérarchie et la proportion des données, utile pour repérer des déséquilibres ou des dominances dans les jeux de données, simple d'utilisation.

2.3.9 umap-learn

UMAP (Uniform Manifold Approximation and Projection) est une technique de réduction de dimensionnalité non linéaire basée sur la théorie des graphes et de la topologie algébrique. Elle est particulièrement utilisée pour visualiser des données complexes en 2D ou 3D tout en préservant autant que possible la structure locale et globale du jeu de données.

Dans ce projet, UMAP-learn a été utilisé pour réduire les dimensions du jeu de données multivarié issu du système électrique (courants, tensions, composantes symétriques, etc.) afin d'explorer visuellement les regroupements naturels, la séparation entre les types de défauts, ou encore pour détecter des anomalies.

Fonctionnalités clés :

- Réduction de dimension rapide et efficace pour grands jeux de données.
- Supporte les données supervisées (via y) ou non supervisées.
- Préserve à la fois la structure locale (voisinage) et globale (forme des clusters).

Avantages :

- Très utile pour la visualisation (mieux que PCA pour les structures complexes).
- S'intègre facilement avec des bibliothèques comme Seaborn ou Matplotlib.
- Compatible avec scikit-learn (API similaire).

2.4 Outils de gestion de version et stockage : Git, GitHub, GitLab

Dans le cadre d'un projet de développement en intelligence artificielle (IA), la gestion du code source et des versions est un aspect fondamental pour assurer la traçabilité, la collaboration et la gestion des erreurs. **Git** est un système de contrôle de version décentralisé qui permet de suivre en temps réel toutes les modifications effectuées sur le projet. Voici pourquoi **Git**, ainsi que ses plateformes associées **GitHub** et **GitLab**, sont des outils essentiels pour ce projet :

Contrôle de version : Git enregistre l'historique de chaque modification apportée au code, ce qui permet de revenir à une version antérieure si nécessaire, offrant ainsi une grande flexibilité et une sécurité accrue.

Gestion des branches : Chaque fonctionnalité, correction de bug ou test peut être réalisé sur une branche distincte. Cela évite que des changements conflictuels se produisent entre différents membres de l'équipe et permet une gestion fluide du code.

Collaboration fluide : GitHub et GitLab permettent de travailler en collaboration avec plusieurs développeurs en offrant des fonctionnalités comme les **pull requests**, où chaque changement peut être examiné avant d'être intégré au code principal. Cela améliore la qualité du code et permet un retour structuré.

Documentation : Git, via **GitHub**, offre la possibilité de documenter chaque modification du code avec des messages de commit détaillés. Cela permet de suivre l'évolution du projet et d'assurer une meilleure traçabilité des décisions prises.

2.5 Outils de présentation (Website) : Streamlit

Streamlit est un outil fondamental dans la présentation des modèles d'intelligence artificielle (IA) en permettant de créer des applications web interactives avec une rapidité et une facilité exceptionnelles. Ce Framework Python permet de créer des interfaces utilisateurs simples et interactives pour exposer les résultats des modèles IA sans avoir besoin de connaissances approfondies en développement web.

Les avantages majeurs de **Streamlit** dans un projet d'IA sont les suivants :

Simplicité et rapidité : Streamlit permet de transformer rapidement un script Python en une interface utilisateur interactive. Son API est intuitive, ne nécessitant aucune expérience en HTML, CSS ou JavaScript. Cela permet de se concentrer sur le modèle d'IA et ses résultats plutôt que sur le développement de l'interface.

Interactivité en temps réel : L'une des caractéristiques les plus puissantes de Streamlit est sa capacité à intégrer des éléments interactifs tels que des curseurs, des cases à cocher, des menus déroulants, et des barres de recherche, permettant à l'utilisateur de tester les modèles d'IA et de visualiser les prédictions en temps réel. Cette interactivité renforce l'engagement de l'utilisateur et permet d'adapter facilement les paramètres des modèles pendant l'expérimentation.

Visualisation des résultats : Streamlit facilite l'intégration de graphiques, courbes, matrices de confusion, et autres visualisations essentielles pour analyser la performance des modèles. Par exemple, un utilisateur peut charger un jeu de données, ajuster les paramètres du modèle, et immédiatement voir l'impact de ces changements sur les prédictions et les performances.

Exécution en temps réel : Streamlit est conçu pour exécuter des scripts Python à chaque interaction, ce qui le rend particulièrement utile pour les démonstrations de modèles d'IA. Lorsqu'un utilisateur entre de nouvelles données ou modifie les paramètres, l'application réagit en affichant les nouvelles prédictions en temps réel, offrant une expérience dynamique et fluide.

Accessibilité et déploiement : En quelques étapes simples, Streamlit permet de déployer une application web locale ou dans le cloud, ce qui est particulièrement utile pour les projets d'IA qui nécessitent une démonstration rapide et accessible. De plus, les applications peuvent être partagées en ligne avec d'autres parties prenantes sans nécessiter de configurations ou de déploiement web complexe.

Dans le cadre de ce projet, Streamlit a été utilisé pour créer une interface simple et interactive permettant à l'utilisateur de :

- Charger les données d'entrée du système électrique,
- Tester différents modèles d'IA pour la détection des défauts,
- Visualiser les résultats de manière interactive et intuitive, en ajustant facilement les paramètres des modèles et en analysant les sorties des prédictions.

3 Présentation du Dataset

3.1 Constitution du dataset pour l'entraînement des modèles

Le dataset utilisé dans ce projet a été généré à partir d'une simulation d'un réseau de distribution électrique. Cette simulation a été réalisée à l'aide de MATLAB/Simulink, un environnement de développement dédié aux systèmes dynamiques. Dans cette simulation, des défauts électriques de divers types ont été introduits pour tester la capacité des modèles d'IA à détecter et classer ces défauts.

Les défauts simulés incluent :

Défaut monophasé : Un défaut qui se produit entre une phase et la terre.

Défaut biphasé : Un défaut entre deux phases du réseau.

Défaut triphasé : Un défaut affectant les trois phases.

Court-circuit : Un défaut important causant une interruption brutale.

Les données ont été collectées sous forme de mesures de tension et de courant dans les différentes phases du système de distribution, et ces mesures ont été associées à des labels indiquant le type de défaut.

3.2 Contenu du dataset

Le dataset utilisé dans ce projet contient des enregistrements issus d'un système de distribution électrique triphasé. Chaque ligne représente une observation unique à un instant donné, caractérisée par les valeurs de courant et de tension dans les trois phases (A, B, C), ainsi qu'un type de défaut associé.

❖ Variables (features)

Les principales colonnes du dataset sont :

Ia, Ib, Ic : Valeurs du courant électrique dans les phases A, B et C.

V_a, V_b, V_c : Valeurs de la tension électrique dans les phases A, B et C.

Type de défaut : Étiquette représentant le type de défaut détecté ou simulé.

❖ Classes de défauts

0 = Aucun défaut (No Fault)

1 = Défaut ligne à la terre (LG)

2 = Défaut entre phases A et B (LL AB)

3 = Défaut entre phases B et C (LL BC)

4 = Défaut double phases + terre (LLG)

5 = Défaut triphasé (LLL)

6 = Défaut triphasé + terre (LLLG)

3.3 Prétraitement des données

Avant d'entraîner les modèles d'intelligence artificielle, un prétraitement rigoureux des données a été effectué afin d'assurer la qualité et la cohérence du dataset. Les principales étapes sont décrites ci-dessous :

❖ Nettoyage des données

Les valeurs manquantes ou aberrantes ont été identifiées et supprimées pour éviter d'introduire des biais ou des erreurs pendant l'entraînement. Ce nettoyage garantit l'intégrité des informations utilisées par les algorithmes.

❖ Normalisation

Les variables numériques, notamment les courants (I_a, I_b, I_c) et les tensions (V_a, V_b, V_c), ont été normalisées dans une plage de valeurs entre 0 et 1. Cette étape est cruciale pour que tous les attributs aient le même ordre de grandeur, ce qui améliore la stabilité et la convergence des modèles d'apprentissage.

❖ Encodage des étiquettes

Les étiquettes représentant les types de défauts ont été encodées sous forme numérique. Chaque classe de défaut a reçu un identifiant unique, facilitant ainsi la classification automatique par les modèles. Par exemple :

LG → 1

LL AB → 2

LL BC → 3

LLG → 4

LLL → 5

LLLG → 6

❖ Division du dataset

Le dataset a été réparti en trois sous-ensembles afin de former, valider et tester les modèles de manière équilibrée :

Ensemble d'entraînement (60%) : utilisé pour ajuster les paramètres internes du modèle.

Ensemble de validation (20%) : permet de régler les hyperparamètres et d'éviter le surapprentissage.

Ensemble de test (20%) : utilisé en phase finale pour évaluer objectivement les performances du modèle.

3.4 Analyse exploratoire des données

Avant de passer à l'entraînement des modèles, une analyse exploratoire des données a été effectuée pour mieux comprendre les caractéristiques des défauts simulés et observer la distribution des variables. Les principales étapes de cette analyse ont inclus :

- ❖ **Visualisation des courbes de tension et de courant** : Des graphiques ont été générés pour observer les tendances des signaux avant et après l'apparition des défauts. Ces visualisations ont permis de mieux comprendre comment chaque type de défaut influence les signaux électriques.
- ❖ **Distribution des types de défauts** : Un histogramme a été créé pour visualiser la répartition des différents types de défauts dans le dataset. Cela a permis de vérifier que le dataset était équilibré et représentait bien toutes les classes de défauts.

- ❖ **Corrélations entre les variables :** Une analyse de corrélation a été réalisée pour identifier les relations entre les différentes mesures de courant et de tension, ainsi que l'impact de ces relations sur la classification des défauts.

3.5 Electrical fault Dataset Analysis Techniques

3.5.1 distribution des fonctionnalités d'entrée importées

La phase d'analyse exploratoire des données du projet. Il permet de visualiser la distribution des principales caractéristiques électriques mesurées, à savoir les courants (I_a , I_b , I_c) et les tensions (V_a , V_b , V_c), présentes dans le fichier `classData.csv`. En combinant les colonnes de défauts (G, C, B, A) pour créer un type de défaut global (`fault_type`), le code prépare les données à l'analyse. Les histogrammes générés pour chaque variable offrent une vue d'ensemble sur leur comportement statistique, mettent en évidence les éventuelles valeurs extrêmes et permettent d'évaluer la qualité des données. Ces visualisations sont importantes pour prendre des décisions éclairées lors du prétraitement, comme la normalisation ou la sélection des variables les plus pertinentes pour l'entraînement d'un modèle d'intelligence artificielle.

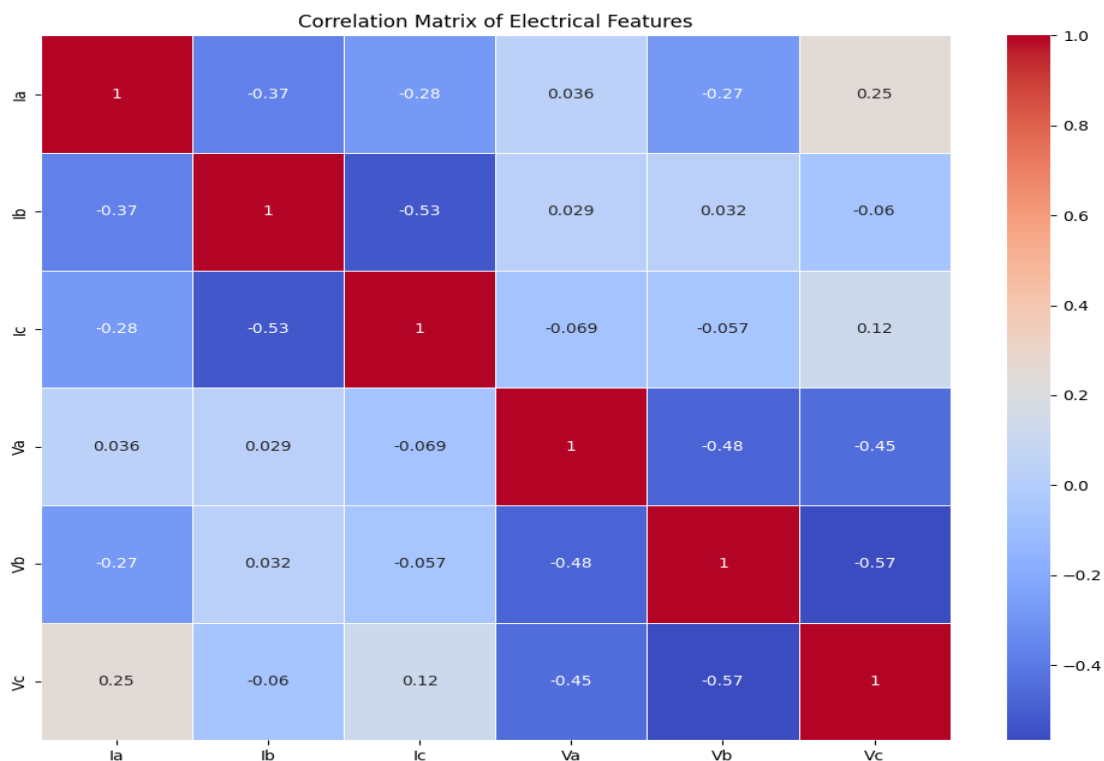


Figure 1 : Distribution des caractéristiques d'entrée (courants et tensions)

3.5.2 Carte thermique de corrélation

Génère une matrice de corrélation pour évaluer les relations linéaires entre les différentes caractéristiques électriques du jeu de données, notamment les courants (I_a , I_b , I_c) et les tensions (V_a , V_b , V_c). La corrélation permet d'identifier les variables qui évoluent ensemble, ce qui est utile pour détecter les redondances ou les dépendances fortes entre les attributs. Le code utilise la bibliothèque Seaborn pour afficher la matrice sous forme de carte thermique (heatmap), avec des couleurs allant du bleu au rouge selon l'intensité de la corrélation. Les coefficients de corrélation sont également affichés pour faciliter l'interprétation. Cette étape est importante dans le processus de sélection de caractéristiques, car elle permet de repérer les variables qui peuvent être combinées, ignorées ou transformées pour améliorer la performance des modèles d'intelligence artificielle.

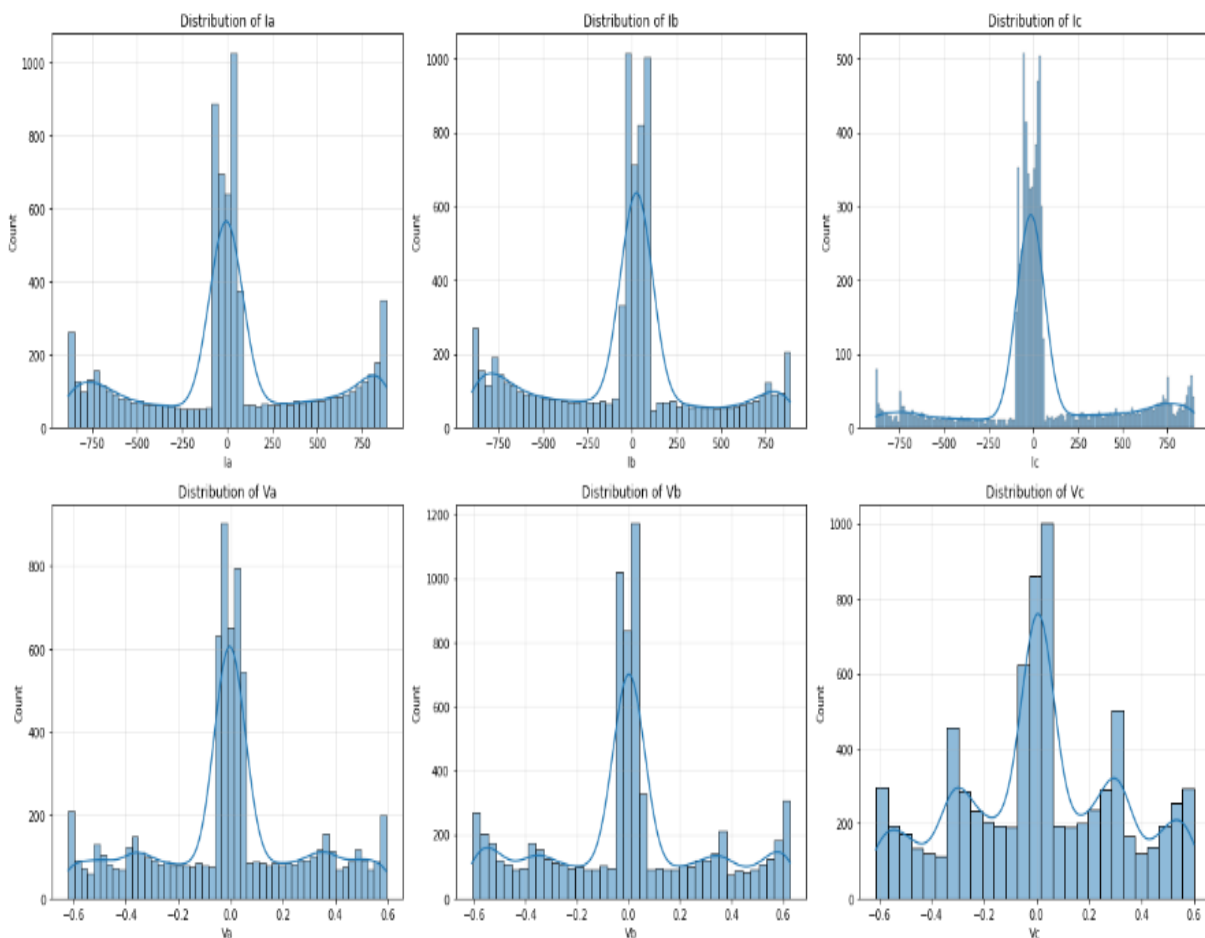


Figure 2 : Carte thermique de corrélation entre les variables électriques

3.5.3 Répartition par type de falaut

Effectue une analyse approfondie de la distribution des caractéristiques électriques (courants et tensions) en fonction des différents types de défauts présents dans le système de distribution. En utilisant des boxplots, il permet de visualiser la répartition des données pour chaque variable en fonction du type de défaut, en affichant les médianes, les quartiles et les valeurs aberrantes. Cela aide à mieux comprendre la variabilité des caractéristiques en lien avec chaque catégorie de défaut (G, C, B, A). L'objectif principal de cette analyse est de discerner les différences notables entre les types de défauts et d'évaluer la capacité des variables à discriminer efficacement ces classes. Cette étape est cruciale pour la sélection des variables les plus informatives et pertinentes pour l'entraînement des modèles de classification, assurant ainsi une meilleure performance prédictive du système global de détection des défauts.

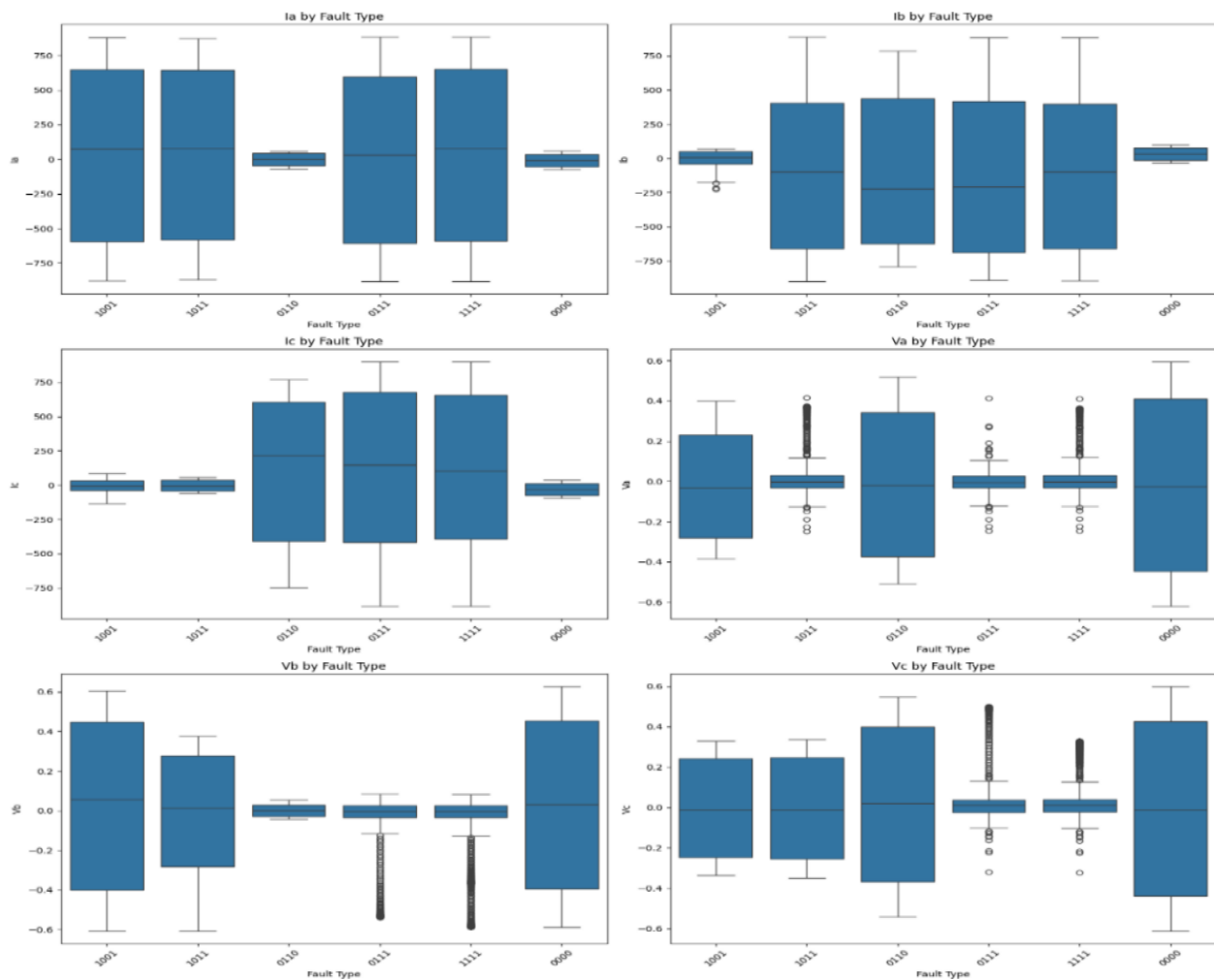


Figure 3 : Répartition des types de défauts dans le dataset

3.5.4 Visualisation PCA avec types de défauts

L'Analyse en Composantes Principales (PCA) pour réduire la dimensionnalité des données d'entrée tout en préservant la variance la plus significative des caractéristiques électriques. Le code commence par normaliser les données, ce qui est essentiel pour garantir que toutes les variables ont une échelle comparable, ce qui est crucial pour l'efficacité de la PCA. Ensuite, il applique la PCA en réduisant les données à deux composantes principales (PC1 et PC2), permettant ainsi une projection bidimensionnelle des données. Cette visualisation facilite l'observation des relations entre les différents types de défauts (G, C, B, A) en projetant les points dans un espace réduit. L'objectif est d'explorer visuellement la manière dont les types de défauts se distinguent dans cet espace à deux dimensions, ce qui peut aider à identifier des structures sous-jacentes ou des patterns dans les données. La variance expliquée par chaque composante est également affichée pour donner une idée de l'importance de chaque composante dans la représentation des données. Cette étape est cruciale pour évaluer si la réduction de dimensionnalité peut améliorer les performances des modèles de classification tout en conservant l'essentiel des informations.

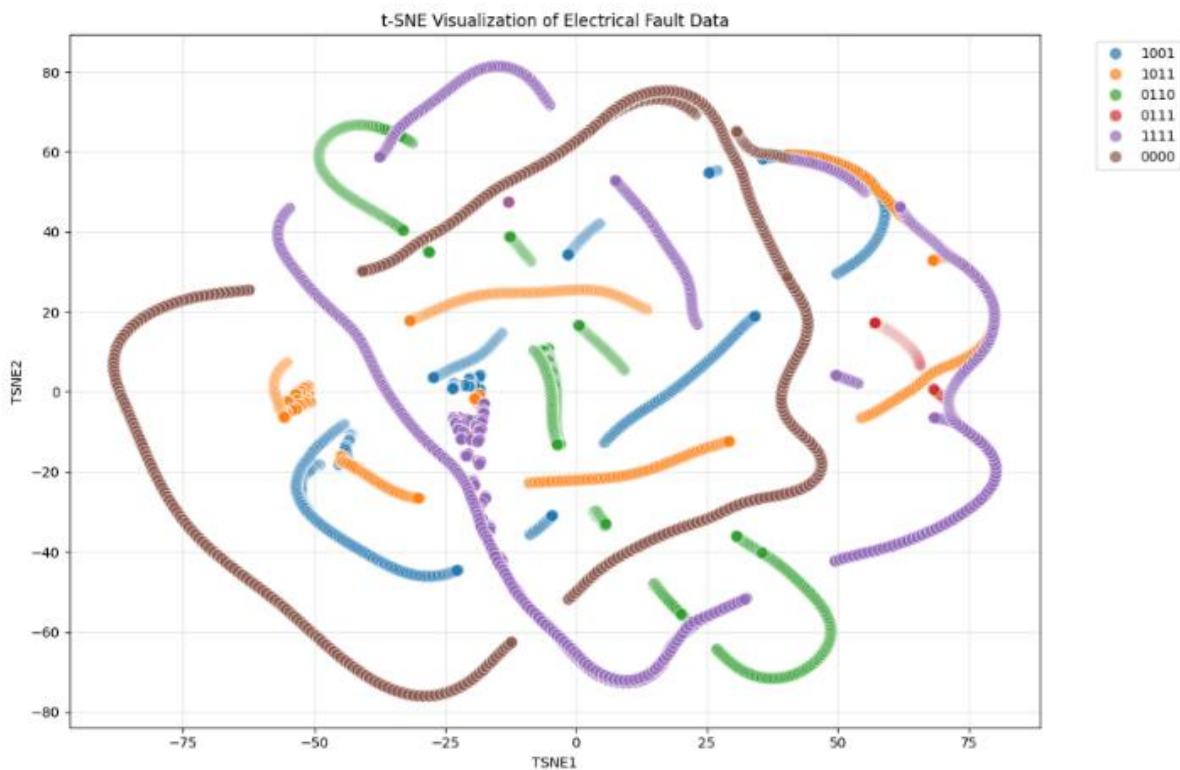


Figure 4 : Projection PCA (2D) des types de défauts

3.5.5 Visualisation des données avec t-SNE

t-SNE (t-distributed Stochastic Neighbor Embedding), une technique de réduction de dimensionnalité non linéaire, pour visualiser les données d'entrée dans un espace de 2 dimensions. Contrairement à la PCA, qui est une méthode linéaire, le t-SNE est particulièrement efficace pour capturer des relations complexes entre les données et révèle des structures locales plus subtiles dans les jeux de données à haute dimension. Le code commence par appliquer la normalisation des données, afin que chaque variable soit sur une échelle comparable. Ensuite, le t-SNE est appliqué pour réduire les données à deux composantes principales (TSNE1 et TSNE2), permettant de les visualiser dans un graphique 2D. Cette visualisation permet d'examiner comment les différents types de défauts (G, C, B, A) se répartissent et se séparent dans l'espace réduit, offrant ainsi des insights sur la capacité de t-SNE à discerner les classes dans les données. L'objectif est de mieux comprendre la structure interne des données et de vérifier si les classes sont bien séparées, ce qui pourrait indiquer que les modèles de classification pourront facilement distinguer ces classes.

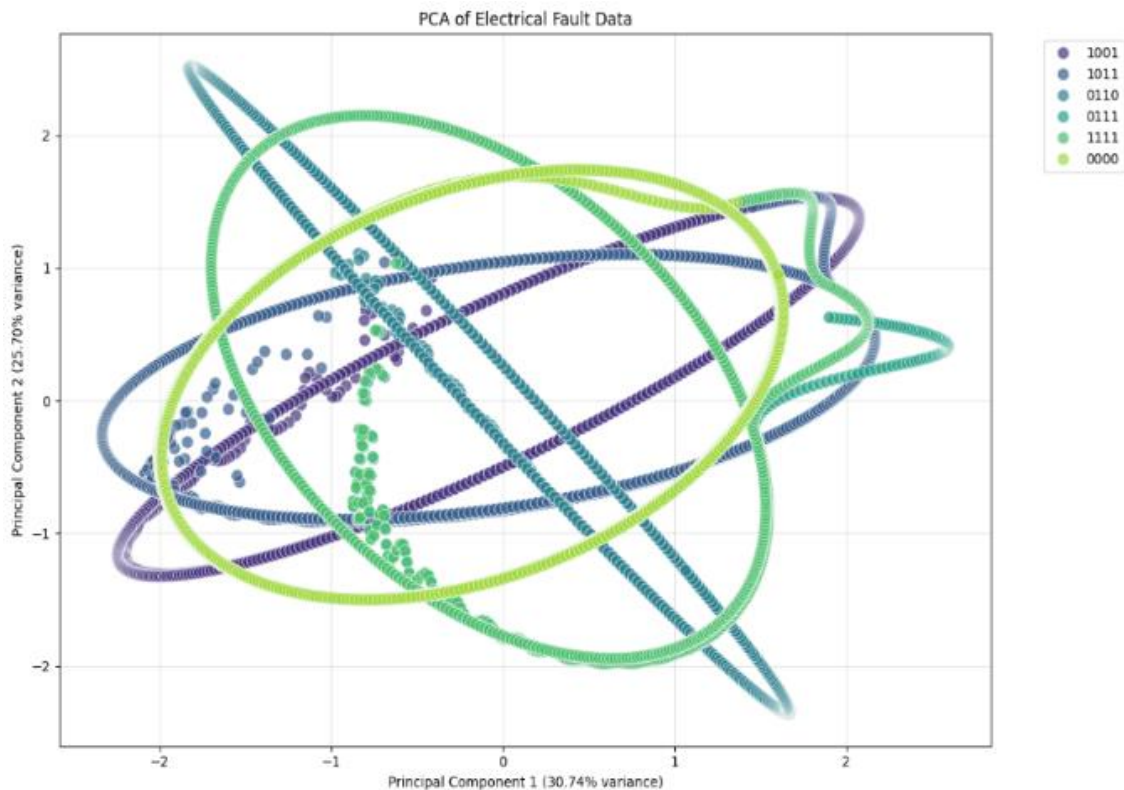


Figure 5 : Visualisation des données avec t-SNE

3.5.6 Visualisation de l'analyse de phase

Phase pour visualiser la relation entre les courants (I_a , I_b) et les tensions (V_a , V_b) dans un système triphasé. Il utilise des nuages de points (scatter plots) pour représenter les données de phase entre deux variables à la fois : I_a par rapport à I_b pour les courants et V_a par rapport à V_b pour les tensions. Chaque point est coloré en fonction du type de défaut (G, C, B, A) pour aider à observer comment les différentes classes de défauts se comportent par rapport aux autres caractéristiques du système. Ces graphiques sont particulièrement utiles pour analyser la symétrie, les anomalies et les relations entre les différentes phases dans un système électrique, en offrant une vue d'ensemble sur l'interdépendance entre les courants et tensions sous différents types de défauts. Ces visualisations permettent d'identifier d'éventuelles caractéristiques distinctives liées à chaque type de défaut, ce qui peut aider à la classification dans un modèle d'intelligence artificielle.

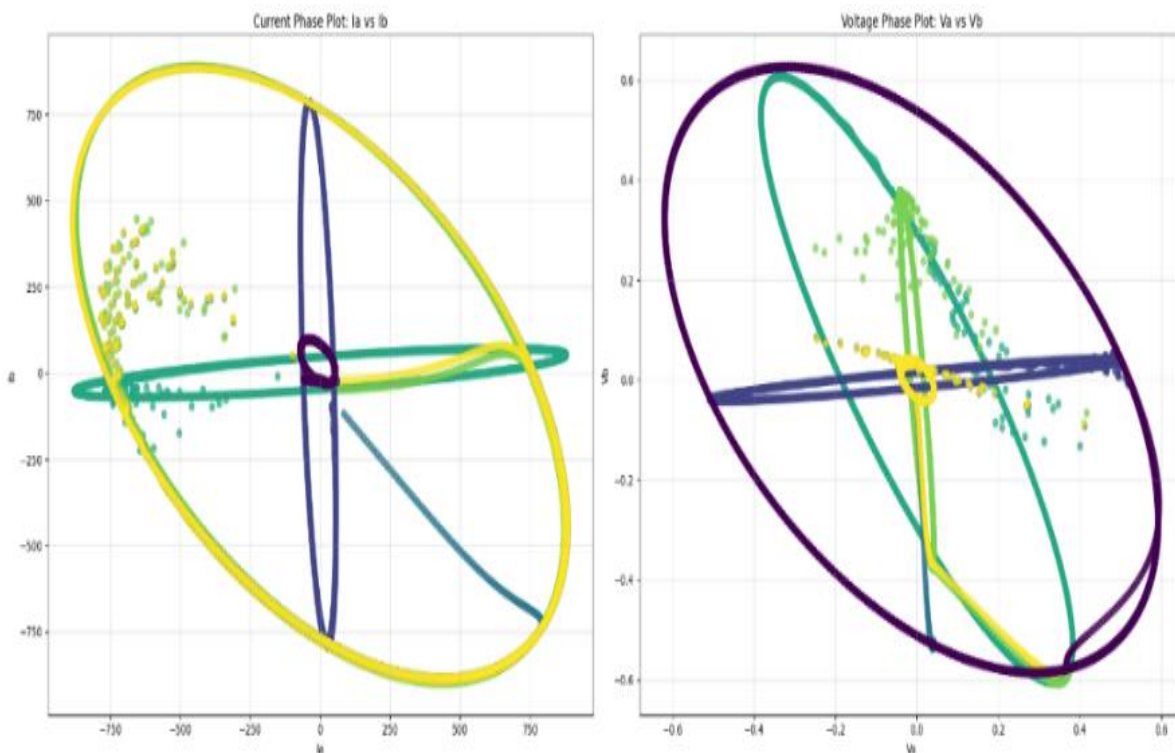


Figure 6 : Scatter plot des phases de courant et de tension (I_a vs I_b , V_a vs V_b)

3.5.6.1 Tracé de coordonnées parallèles

Un graphique en coordonnées parallèles pour visualiser les relations entre les différentes caractéristiques électriques (courants et tensions) en fonction des types de défauts (G, C, B, A). Le graphique en coordonnées parallèles est une technique efficace pour explorer des données multidimensionnelles, en représentant chaque échantillon sous forme de lignes qui traversent

plusieurs axes représentant les caractéristiques. Chaque ligne est colorée selon le type de défaut pour permettre une distinction visuelle des différentes classes. Avant de créer ce graphique, les données sont échantillonnées pour éviter de surcharger le graphique avec trop d'échantillons, puis normalisées pour garantir que toutes les variables sont sur la même échelle. Ce graphique est particulièrement utile pour observer les relations entre les variables dans des dimensions élevées, en mettant en évidence les motifs ou anomalies qui peuvent exister entre les types de défauts. Il permet de repérer visuellement comment les différents types de défauts se comportent par rapport aux autres caractéristiques du système, offrant ainsi un aperçu précieux pour la sélection des variables et la classification.

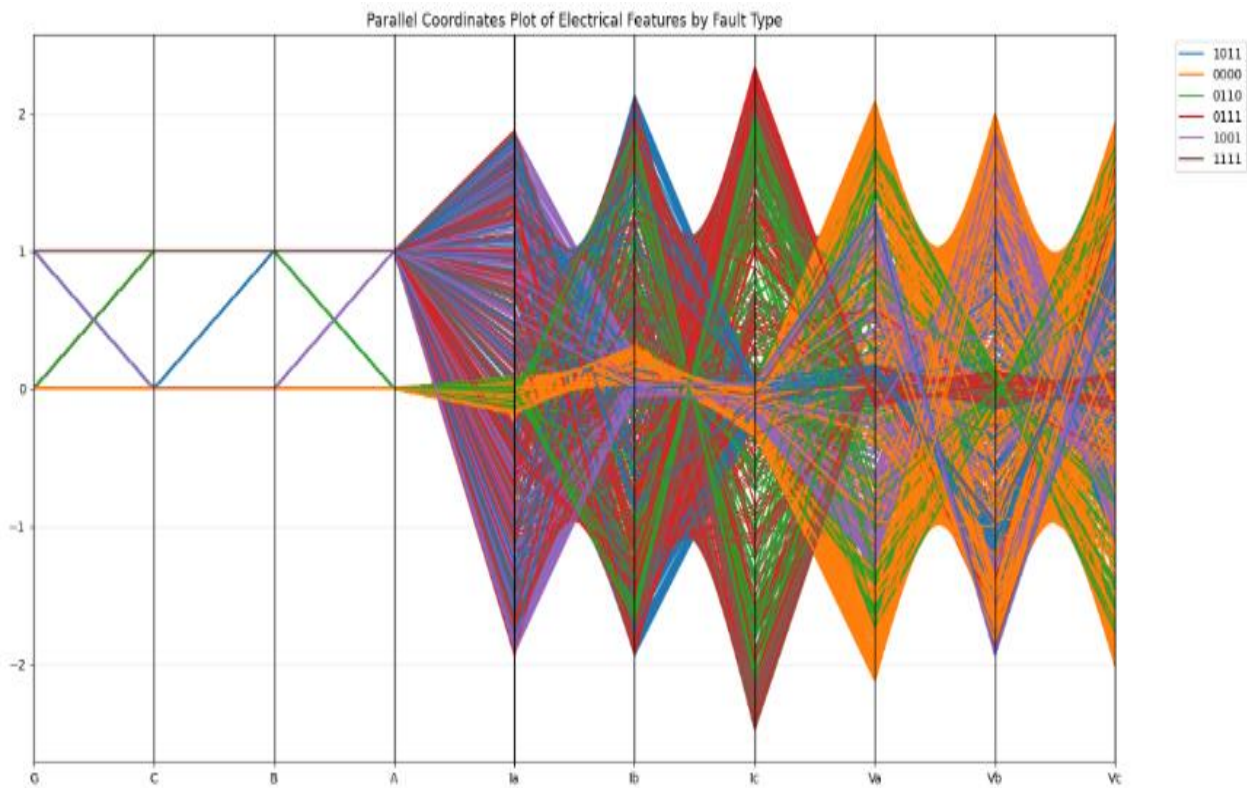


Figure 7 : Graphique en coordonnées parallèles selon les types de défauts

3.5.7 Analyse de clustering et de silhouette

Analyse de clustering pour déterminer le nombre optimal de clusters dans les données électriques en utilisant l'algorithme K-Means. L'objectif principal est de trouver le nombre de clusters qui maximisera la séparation entre les groupes tout en minimisant la variance à l'intérieur de chaque groupe. Le code varie le nombre de clusters entre 2 et 10 et calcule pour chaque cas le score de silhouette, une mesure de la qualité du clustering. Un score de silhouette élevé indique que les échantillons sont bien séparés des autres clusters tout en étant proches de ceux du même cluster. Les résultats sont tracés sous forme de graphique pour visualiser l'évolution du score de

silhouette en fonction du nombre de clusters. Cette analyse est essentielle pour choisir le nombre optimal de clusters, un choix important qui influencera la performance du modèle de clustering dans la détection des défauts électriques.

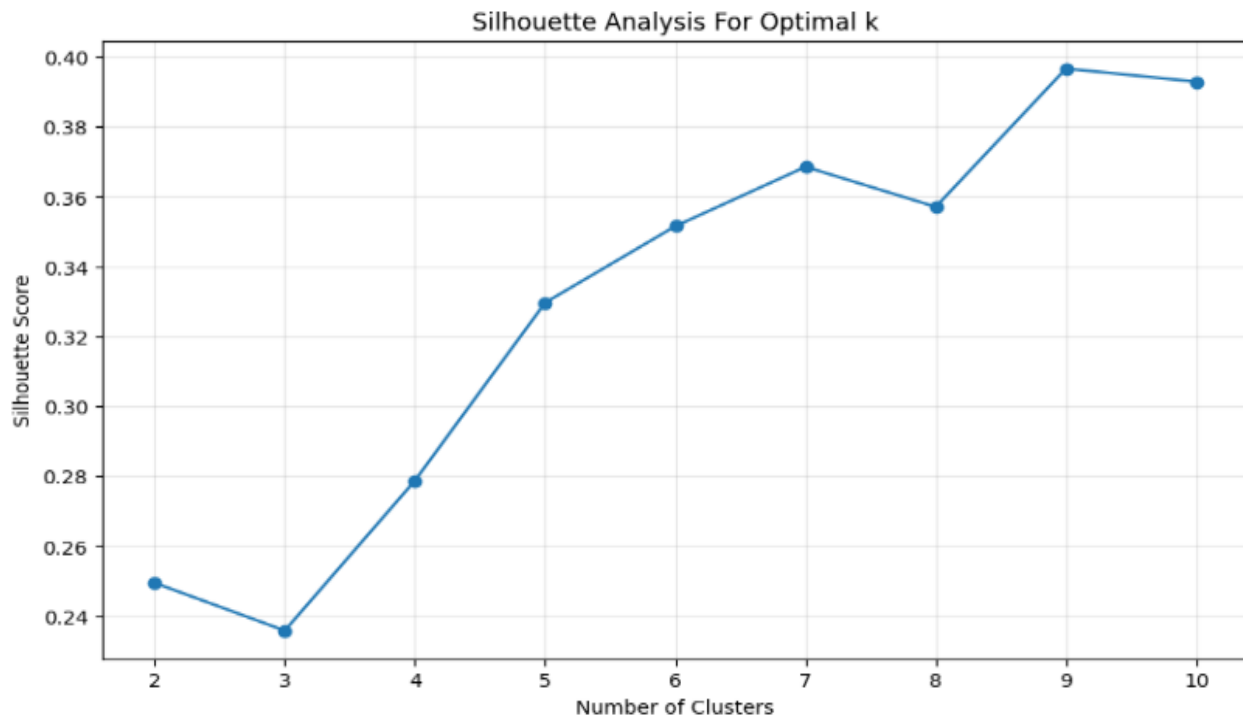


Figure 8 : Score de silhouette selon le nombre de clusters (KMeans)

3.5.8 Visualisation PCA 3D

L'Analyse en Composantes Principales (PCA) pour réduire la dimensionnalité des données normalisées à trois composantes principales, permettant ainsi une visualisation tridimensionnelle des défauts électriques. La PCA permet de projeter les données d'origine dans un espace à 3 dimensions tout en conservant le maximum de variance possible. Cela facilite la représentation visuelle de la structure des données et la différenciation entre les types de défauts. Chaque point du nuage 3D représente une instance de données projetée sur les trois composantes principales (PC1, PC2, PC3), et est coloré selon son type de défaut. Cette visualisation est utile pour détecter visuellement les regroupements naturels, les similarités et les séparations entre les différentes classes de défauts.

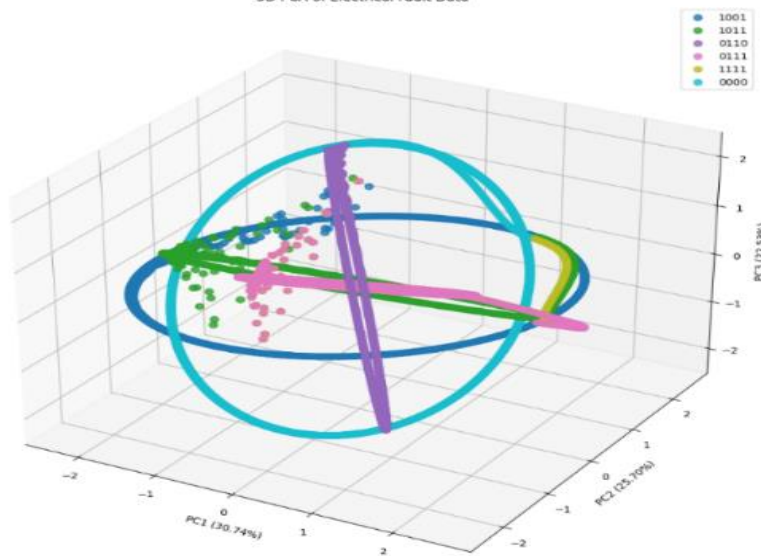


Figure 9 : Visualisation PCA en 3D des défauts électriques

3.5.9 Diagramme polaire du courant et des tensions triphasés

Représenter les courants et tensions triphasés (I_a , I_b , I_c et V_a , V_b , V_c) sous forme de diagrammes polaires pour différents types de défauts électriques. Chaque défaut est représenté par une figure où les trois phases sont positionnées à des angles fixes (0° , 120° , 240°), correspondant aux positions relatives naturelles des phases dans un système triphasé. Pour chaque type de défaut, les valeurs moyennes des courants et tensions sont calculées, puis tracées dans un graphique polaire fermé, ce qui met en évidence l'équilibre ou le déséquilibre entre les phases. Ces diagrammes sont très utiles pour visualiser les anomalies et asymétries caractéristiques de certains défauts.

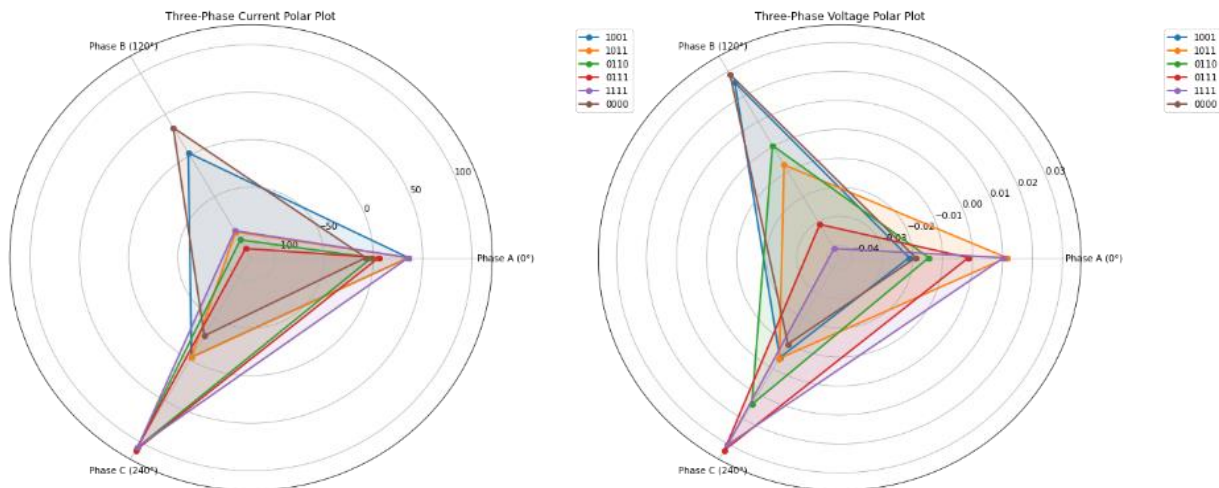


Figure 9 :

3.5.10 Importance des fonctionnalités à l'aide de la forêt aléatoire

Objectif de **calculer et analyser les composantes symétriques** des courants et tensions triphasés mesurés dans un système électrique soumis à différents types de défauts (LG, LL, LLG, etc.). À partir de données brutes (I_a , I_b , I_c , V_a , V_b , V_c), il applique la transformation de Fortescue pour décomposer les signaux en **composantes de séquence zéro, positive et négative**. Ces composantes permettent de mieux identifier et caractériser la nature des défauts électriques. Le code utilise ensuite des représentations graphiques (boxplots) pour visualiser la variation de chaque composante en fonction du type de défaut, ce qui facilite l'interprétation des résultats et peut servir à l'entraînement de modèles d'intelligence artificielle pour la **détection ou classification automatique des défauts** dans les réseaux de distribution.

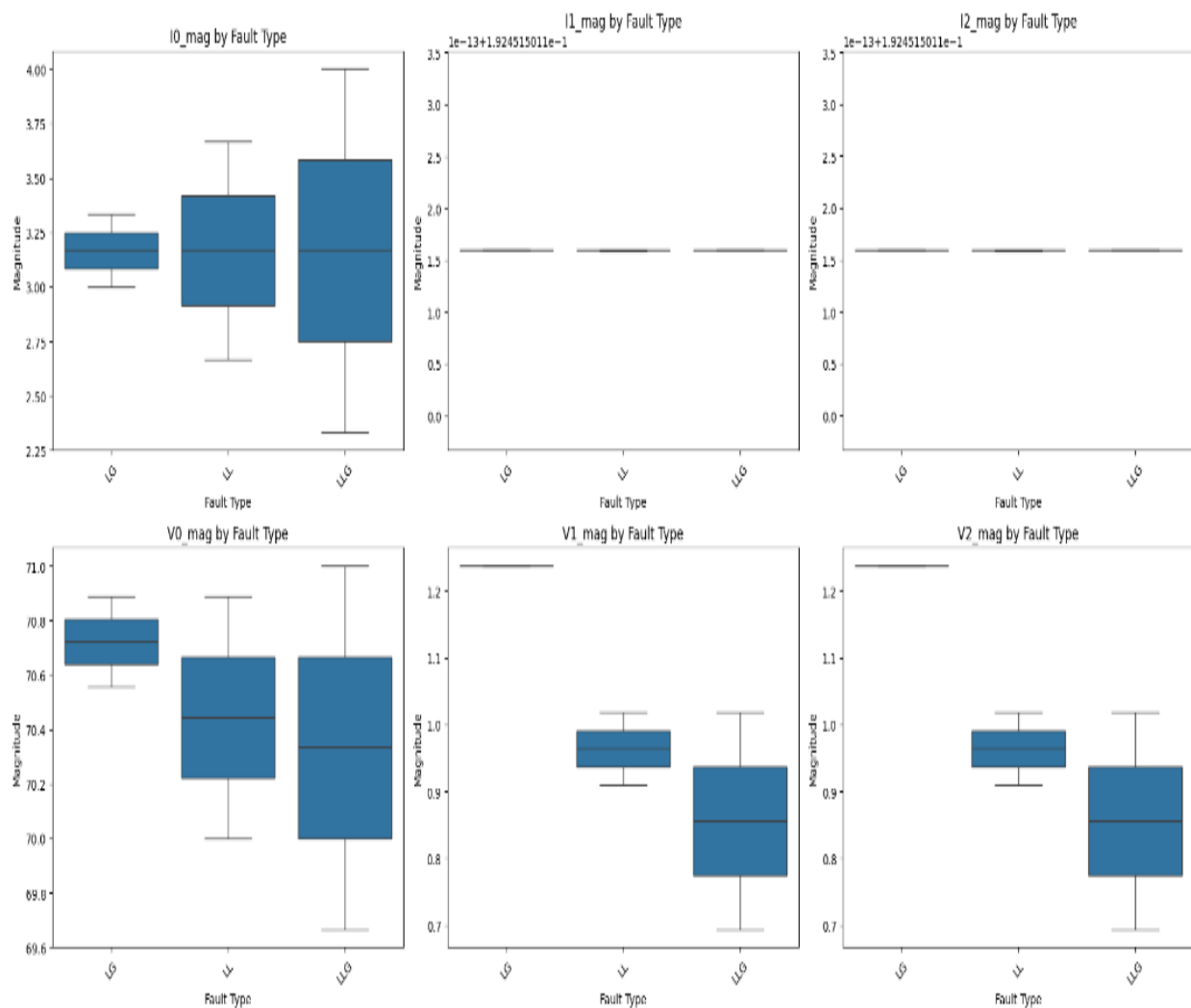


Figure 10 : Diagramme polaire des tensions et courants triphasés

3.5.11 Analyse des composantes symétriques (critique pour les défauts électriques)

calcule les composantes symétriques des courants et tensions triphasés, à savoir les composantes zéro (I_0/V_0), positive (I_1/V_1) et négative (I_2/V_2), en utilisant une transformation mathématique standard dans les systèmes triphasés. La méthode repose sur la matrice de transformation de Fortescue. Ces composantes permettent d'isoler les différents déséquilibres présents dans le système, ce qui est essentiel pour la détection et la classification des défauts électriques.

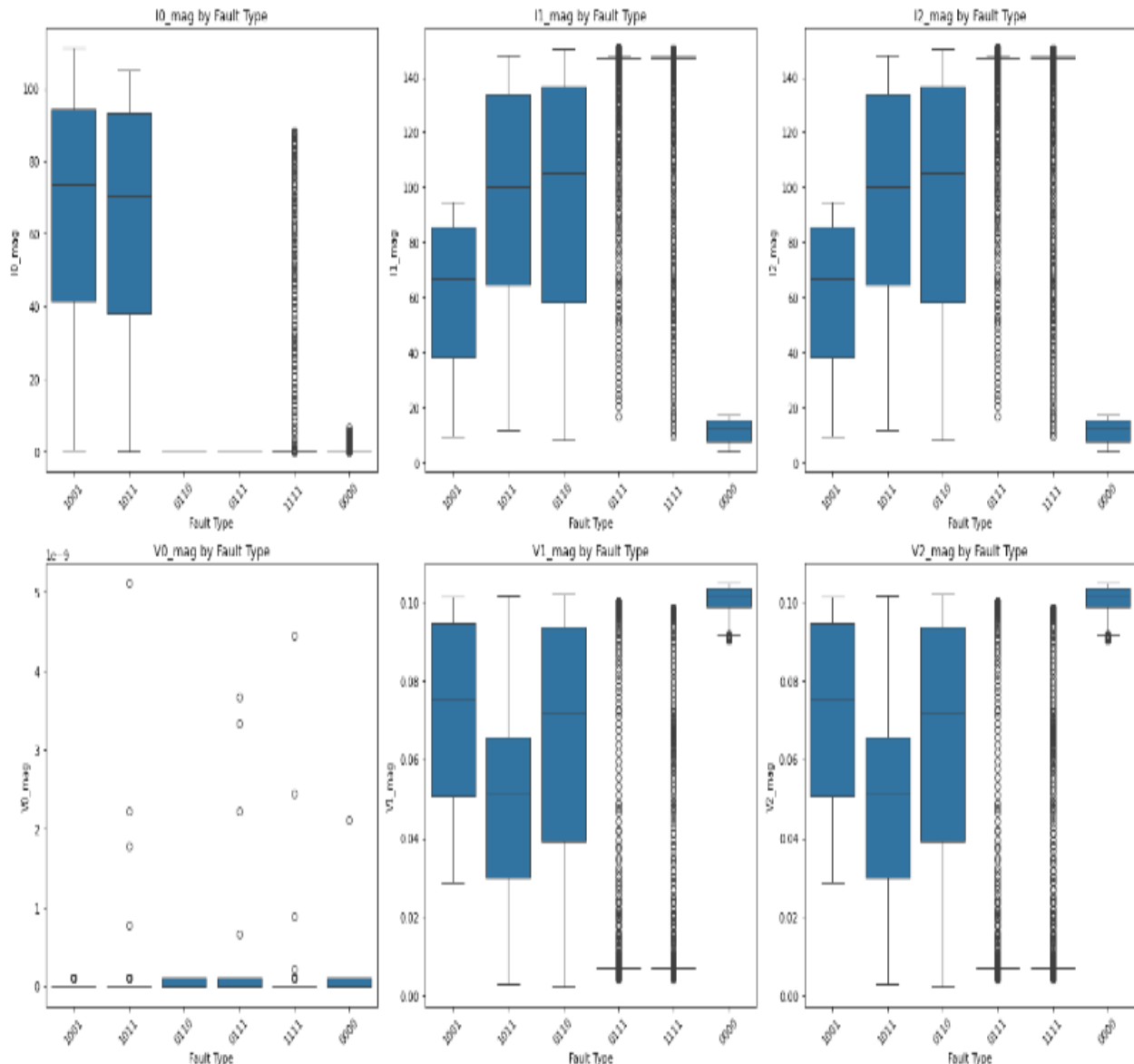


Figure 11 : Boxplots des composantes symétriques selon les défauts

3.5.12 répartition des types de pannes

Permet de représenter graphiquement la distribution des types de défauts électriques enregistrés dans un réseau. Il commence par un diagramme en barres qui affiche le nombre d'occurrences de chaque type de défaut (par exemple : court-circuit phase-terre, entre phases, etc.), ce qui donne une vue claire sur les défauts les plus fréquents. Ensuite, un diagramme circulaire est généré pour illustrer la proportion relative de chaque type de défaut dans l'ensemble des données. Ces visualisations sont essentielles pour l'analyse statistique préalable, car elles permettent d'identifier les déséquilibres éventuels dans les données et de mieux comprendre la nature des incidents rencontrés dans le système électrique étudié.

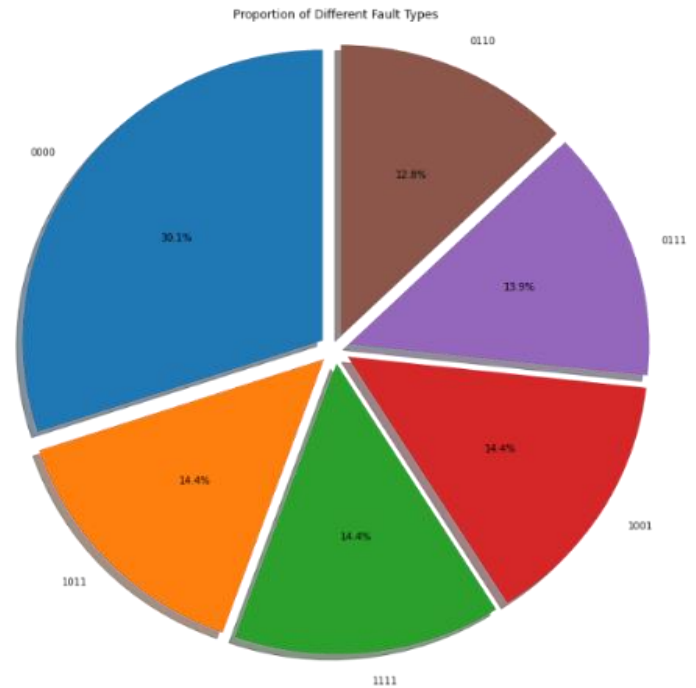


Figure 12 : Analyse des composantes symétriques (I_0 , I_1 , I_2 / V_0 , V_1 , V_2)

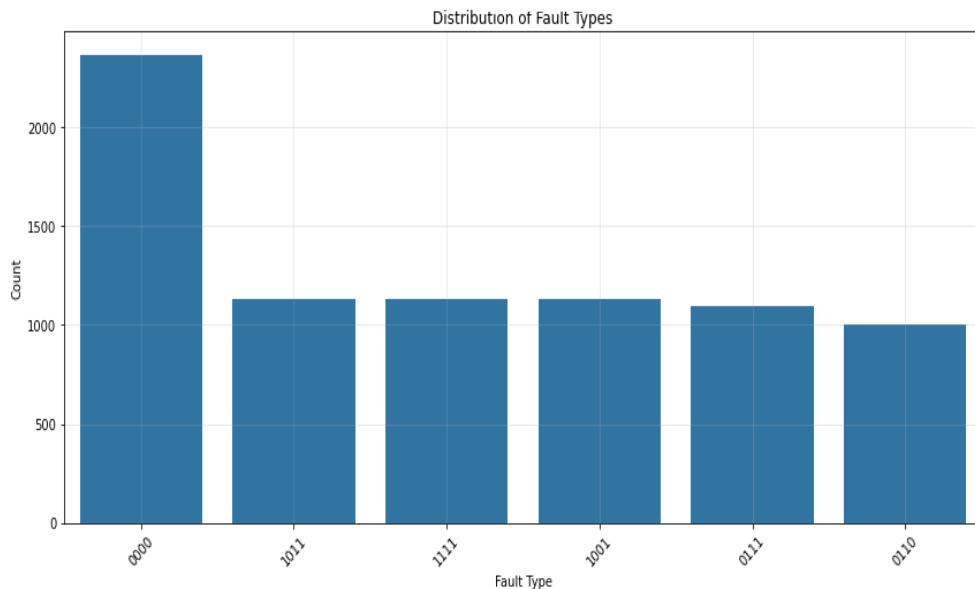


Figure 13 : Histogramme + diagramme circulaire des défauts simulés

3.5.13 clustering non supervisé vs types de défauts réels

Réalise une analyse de clustering non supervisée à l'aide de l'algorithme KMeans, dans le but d'examiner dans quelle mesure les groupes formés automatiquement par l'algorithme correspondent aux types de défauts réels présents dans les données.

Les étapes principales sont :

1. **Clustering avec KMeans** : le modèle est entraîné pour identifier k groupes (où k est le nombre réel de types de défauts).
2. **Étiquetage des défauts réels** : les types de défauts sont encodés sous forme numérique pour permettre la comparaison.
3. **Matrice de confusion** : elle compare la correspondance entre les clusters prédits et les classes réelles, ce qui permet de visualiser si certains clusters recourent bien certaines classes.
4. **Calcul de la pureté** : un indicateur global de performance des clusters, mesurant le pourcentage d'échantillons correctement regroupés par rapport à la classe majoritaire dans chaque cluster.

Cette approche est utile pour :

- Évaluer la structure naturelle des données sans supervision,
- Vérifier si les caractéristiques discriminantes permettent de retrouver les classes de défauts,
- Diagnostiquer la qualité des regroupements obtenus sans avoir recours à l'étiquetage

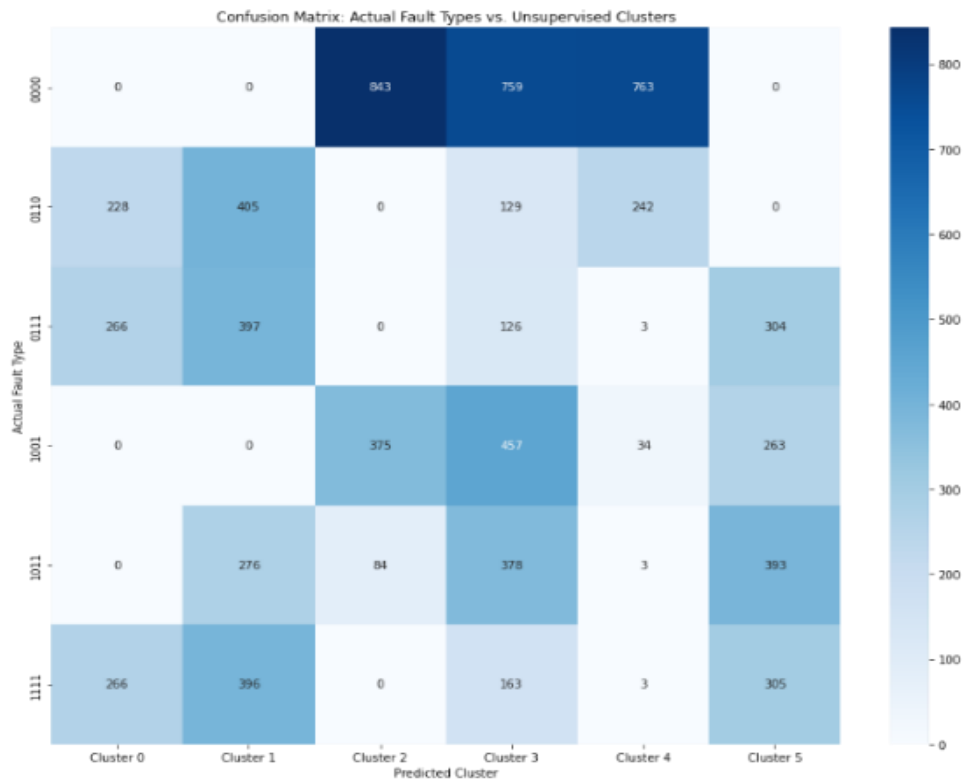


Figure 14 : Matrice de confusion pour le clustering non supervisé

3.5.14 Représentation vectorielle d'un système triphasé

Permet de visualiser le comportement vectoriel des courants et tensions triphasés à travers des diagrammes en coordonnées polaires, également appelés *diagrammes vectoriels*.

Les principales étapes sont :

1. **Sélection d'échantillons représentatifs** pour chaque type de défaut, afin de montrer une variété de comportements dans le système électrique.
2. **Assomption des angles de phase** (0° , 120° , 240°) pour les phases A, B et C, en l'absence de données de phase réelles.
3. **Construction de flèches polaires** :
 - Chaque courant (I_a , I_b , I_c) et tension (V_a , V_b , V_c) est représenté comme un vecteur à partir de l'origine, avec une amplitude égale à la valeur mesurée et une direction selon l'angle de phase supposé.

- Les couleurs différencient les types de défauts.

4. Création de légendes personnalisées pour identifier les types de défauts sur le diagramme.

cette représentation est précieuse pour :

- Observer les déséquilibres dans les phases.
- Identifier visuellement les modifications caractéristiques des vecteurs selon les types de défauts (symétrie, amplitude, etc.).
- Aider à l'interprétation intuitive du comportement triphasé dans des situations normales ou défectueuses.

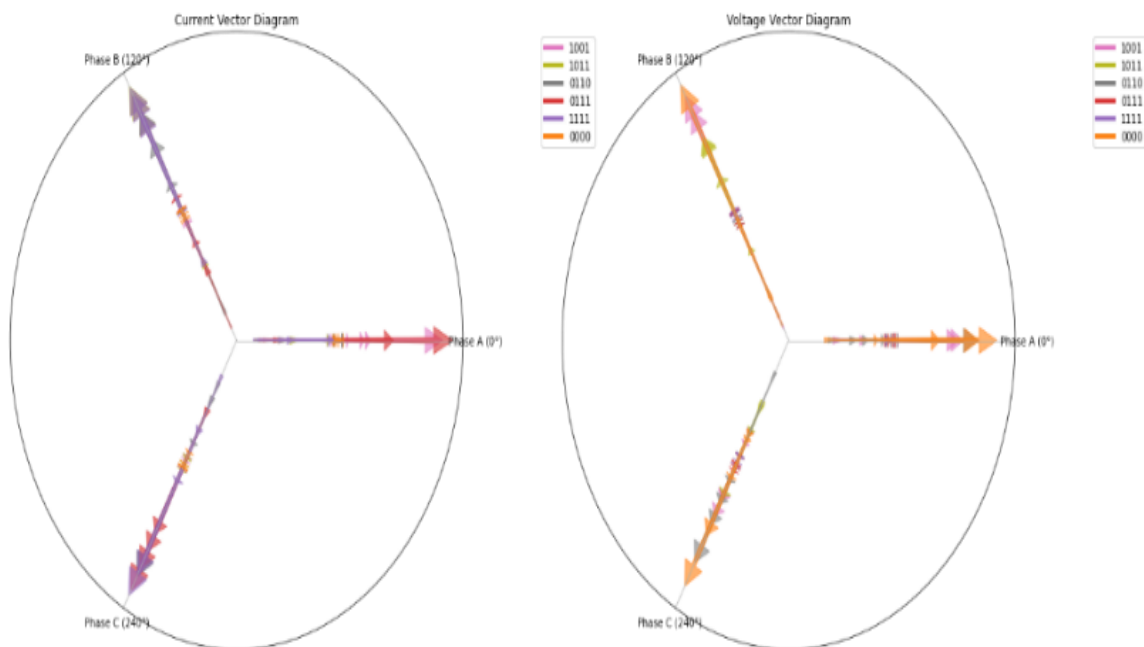


Figure 15 : Représentation vectorielle des défauts triphasés

3.5.15 visualisation de détection d'anomalies

Utilise la méthode d'Isolation Forest, un algorithme non supervisé populaire pour la détection d'anomalies, afin d'identifier les échantillons atypiques dans les données triphasées.

Étapes principales :

1. Détection d'anomalies :

- IsolationForest isole les observations anormales (défauts rares ou atypiques) en construisant des arbres aléatoires.

- Les observations identifiées comme anormales sont marquées avec -1 dans anomaly_score.

2. Réduction de dimension (PCA) :

- Réduction des données à 2 composantes principales (PC1, PC2) pour visualisation 2D.

3. Visualisation :

- Les points normaux sont colorés selon leur type de défaut.
- Les anomalies sont affichées en rouge avec un marqueur distinctif ('x').
- Cela permet de repérer visuellement les zones d'anomalie dans l'espace des caractéristiques.

4. Analyse des anomalies par type de défaut :

- Affichage du nombre d'anomalies détectées par type de défaut sous forme de graphique à barres.
- Cette statistique peut indiquer des types de défauts plus sujets à des comportements irréguliers ou à des erreurs de mesure.

Cette approche est particulièrement utile pour :

Identifier des défauts rares ou non classés.

- Détecter des comportements anormaux dans les réseaux électriques.
- Préparer une phase de nettoyage ou d'analyse approfondie

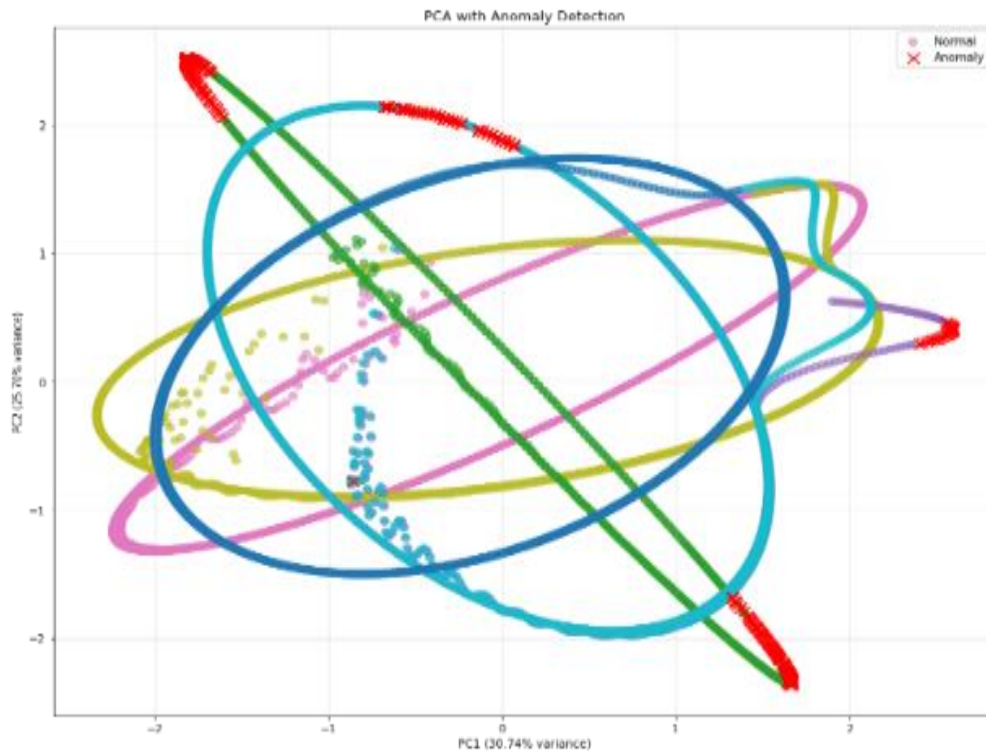


Figure 16 : Détection d'anomalies avec Isolation Forest

3.5.16 Conclusion

L'étude approfondie des données constitue une étape fondamentale dans tout projet d'intelligence artificielle, et ce chapitre en a posé les bases avec rigueur. L'analyse exploratoire réalisée a permis de mettre en lumière les caractéristiques structurelles du dataset, ainsi que les dynamiques internes entre les différentes variables électriques. Des techniques de visualisation avancées, telles que la PCA ou le t-SNE, ont contribué à une compréhension fine des distributions, des corrélations et des éventuelles anomalies.

Le prétraitement des données, incluant la normalisation, la gestion des déséquilibres de classes et la création de nouvelles variables pertinentes, a permis de rendre les données plus exploitables et plus représentatives des conditions réelles du réseau électrique. Ce travail préparatoire, à la fois technique et analytique, constitue un socle robuste pour le développement des modèles d'apprentissage supervisé et non supervisé.

En somme, ce chapitre a assuré une transition fluide entre l'analyse des données brutes et la phase de modélisation. Il a mis en évidence que la qualité des données, tout autant que leur volume, conditionne directement la performance future des algorithmes. Ces fondations solides permettront d'aborder, dans les chapitres suivants, le développement, l'entraînement et l'évaluation des modèles d'intelligence artificielle dans un cadre à la fois fiable et cohérent.

4 Détection des défauts électriques à l'aide de l'IA

4.1 Application du Machine Learning

Le Machine Learning est un sous-ensemble de l'intelligence artificielle qui permet aux systèmes d'apprendre à partir des données sans avoir besoin de programmation explicite. Dans ce projet, plusieurs modèles de Machine Learning ont été testés pour la détection des défauts électriques. Voici les principaux algorithmes utilisés :

4.1.1 K-Nearest Neighbors (KNN)

Le modèle K-Nearest Neighbors (KNN) a été utilisé avec 20 voisins ($n_neighbors = 20$) et la distance euclidienne ($p = 2$). Il a atteint une bonne précision globale de 84,11 %, indiquant une performance satisfaisante dans l'ensemble. Les métriques pondérées comme le F1-score, la précision et le rappel confirment cette efficacité sur la majorité des classes.

Cependant, l'analyse détaillée révèle que les classes 4 et 5 présentent des performances faibles, notamment la classe 5 avec un F1-score très bas (0,36).

L'évaluation est appuyée par des visualisations comme la matrice de confusion, qui montre les erreurs de classification, et la courbe d'apprentissage, qui illustre la manière dont le modèle apprend avec plus de données.

Il est recommandé d'envisager des améliorations ciblées, comme le rééquilibrage des données, le réglage des hyperparamètres (ex. réduire le nombre de voisins), ou l'essai d'algorithmes alternatifs pour mieux traiter les classes sous-performantes.

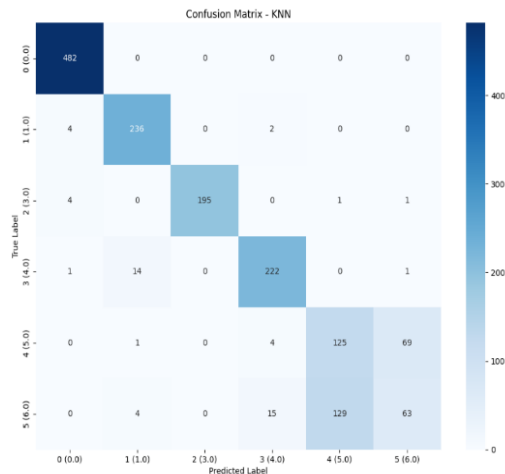


Figure 1

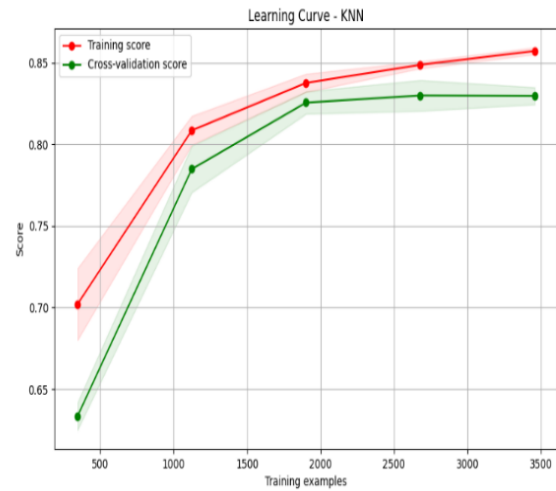


Figure 2 :

4.1.2 Support Vector Machine (SVM)

4.1.2.1 Linear :

Le modèle SVM linéaire a été utilisé avec des paramètres standards ($C=1.0$, $\text{kernel}='linear'$) pour la classification multi-classes.

Il a obtenu une précision globale relativement faible de 61,28 %, indiquant une performance moyenne du modèle sur l'ensemble du jeu de données.

Les métriques pondérées montrent également un F1-score de 0,56, ce qui confirme une efficacité limitée, en particulier sur plusieurs classes.

Les classes 1, 2, 3, 4 et 5 présentent toutes des F1-scores inférieurs à 0,7, avec une attention particulière à porter sur la classe 4, dont le F1-score est extrêmement bas (0,11), signalant une mauvaise capacité de prédiction.

La matrice de confusion et la courbe d'apprentissage révèlent que le modèle a du mal à généraliser correctement sur toutes les classes.

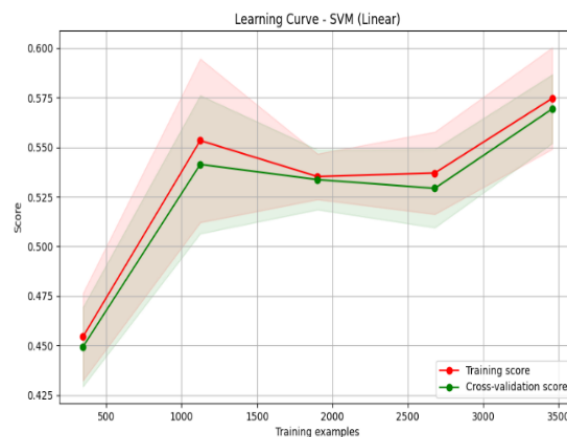
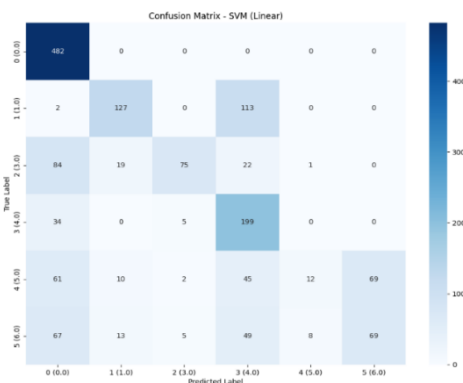


Figure 3

Figure 4

4.1.2.2 RBF :

Le modèle SVM avec noyau RBF a obtenu de bons résultats globaux, avec une précision de 80,61 % et un F1-score pondéré de 0,79. Il s'est montré performant sur la plupart des classes, notamment les classes 0, 1, 2 et 3, qui présentent toutes des F1-scores supérieurs à 0,88.

En revanche, les classes 4 et 5 restent problématiques, avec des F1-scores de 0,59 et 0,24 respectivement, ce qui indique des difficultés à bien les classer.

La matrice de confusion montre une bonne séparation pour les classes dominantes, mais des confusions fréquentes pour les classes minoritaires.

La courbe d'apprentissage suggère un bon comportement général du modèle, sans surapprentissage marqué.

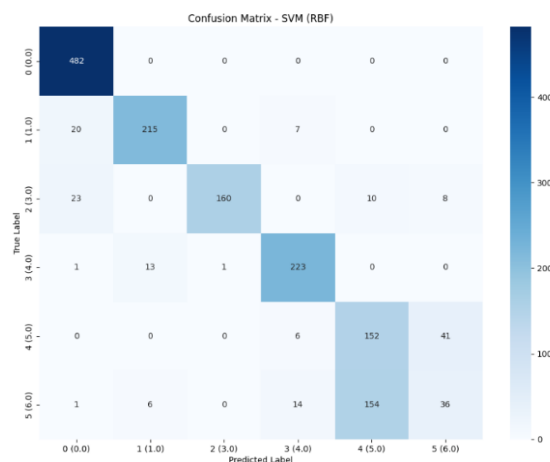


Figure 6 :

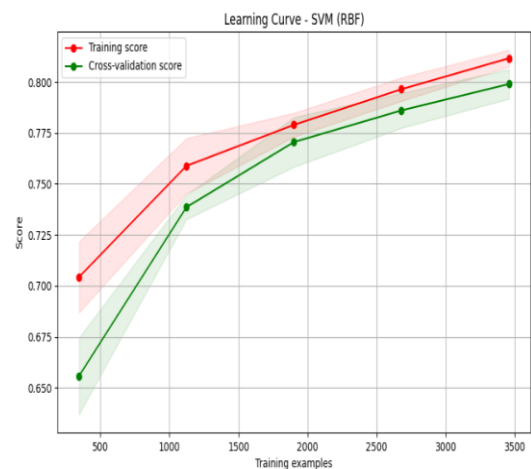


Figure 7 :

4.1.2.3 Optimized :

Le modèle SVM avec noyau RBF optimisé a obtenu les meilleures performances parmi tous les modèles testés, avec une précision de 86,84 % et un F1-score pondéré de 0,8687. Grâce à un ajustement optimal des hyperparamètres ($C=100$, $\gamma=1$), ce modèle parvient à classer correctement presque toutes les classes majoritaires avec des F1-scores supérieurs à 0,98 pour les classes 0 à 3.

Cependant, les classes 4 et 5 présentent toujours des performances inférieures, avec des F1-scores de 0,5383 et 0,4873, respectivement. Ces résultats indiquent une forte performance globale, mais également la nécessité de stratégies spécifiques pour les classes minoritaires ou plus difficiles à

distinguer.

La courbe d'apprentissage montre une bonne convergence et la matrice de confusion révèle peu d'erreurs pour les classes dominantes.

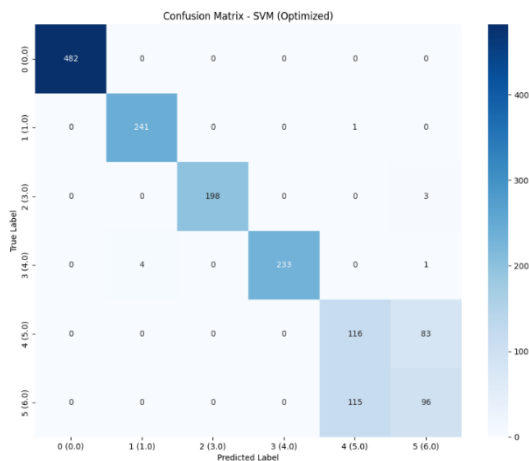


Figure 7 :

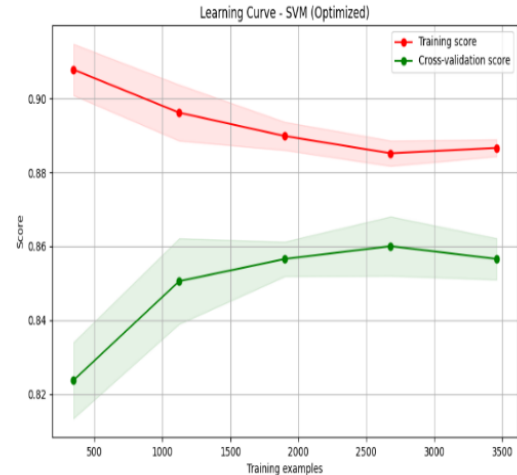


Figure 8 :

4.1.3 Random Forest

Le modèle Random Forest a montré des performances solides avec une précision globale de 85,95 % et un F1-score pondéré de 0,8592. Grâce à ses 100 estimateurs et à sa capacité à gérer des frontières de décision complexes, il a très bien classé les classes majoritaires, atteignant des F1-scores proches ou supérieurs à 0,97 pour les classes 0 à 3.

En revanche, les classes 4 et 5 présentent des résultats plus faibles, avec des F1-scores de 0,5293 et 0,4822, respectivement. Ces résultats suggèrent que, bien que le modèle soit globalement performant, des efforts supplémentaires sont nécessaires pour améliorer la détection des classes minoritaires ou ambiguës.

Le graphe d'importance des variables offre également une bonne visibilité sur les attributs les plus influents, ce qui peut orienter de futures optimisations ou réductions de dimensionnalité.

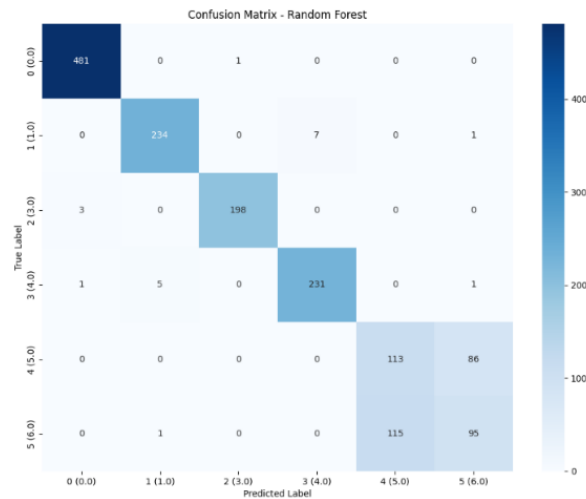


Figure 9 :

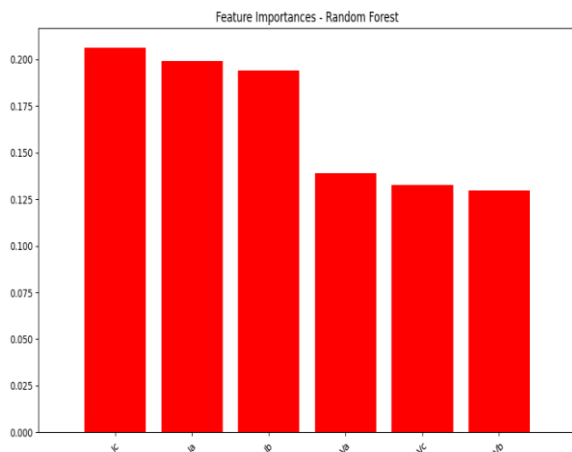


Figure 10 :

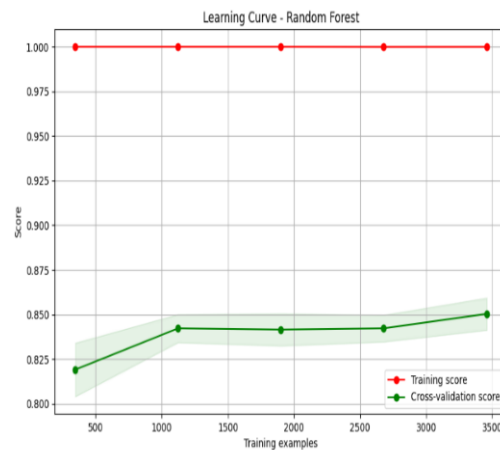


figure 11 :

4.1.4 Arbre de décision (Decision Tree)

Le modèle Decision Tree a obtenu une précision de 81,31 % et un F1-score pondéré de 0,8160, ce qui reflète des performances correctes, bien qu'inférieures à celles du modèle Random Forest. Il a bien classé les classes majoritaires comme les classes 0 à 3, avec des F1-scores supérieurs à 0,84, notamment un excellent score de 0,9793 pour la classe 3.

En revanche, les résultats sont plus faibles pour les classes minoritaires ou plus difficiles à

séparer : la classe 4 a un F1-score de 0,4308, tandis que la classe 5 atteint 0,5357. Cela met en évidence une difficulté à généraliser pour les classes faiblement représentées ou présentant un chevauchement dans l'espace des caractéristiques.

Le modèle offre néanmoins une interprétabilité précieuse, notamment grâce à l'arbre de décision visualisé et à l'analyse d'importance des attributs, ce qui peut faciliter des ajustements manuels ou une exploration plus fine des règles de décision.

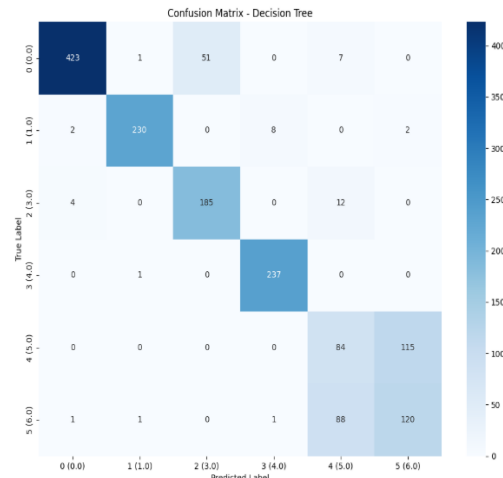


Figure 12:

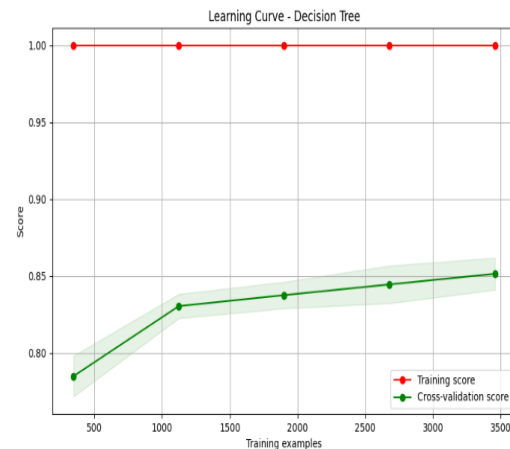


Figure 13 :

4.1.5 Régression logistique

Le modèle Logistic Regression présente des résultats décevants, avec une précision de 37,19 % et un F1-score pondéré de 0,2525, ce qui indique une mauvaise performance globale sur l'ensemble des classes. En particulier, il peine à prédire la majorité des classes, notamment les classes 1, 2, 4 et 5, qui ont des scores F1 égaux à 0. La seule classe ayant un score acceptable est la classe 0, mais cela ne compense pas les échecs sur les autres classes.

Les performances sont particulièrement faibles sur les classes 1, 2, 4 et 5, qui représentent des cas difficiles pour un modèle linéaire. Cela suggère que les relations entre les caractéristiques et ces classes ne sont pas bien capturées par un modèle logistique, souvent plus performant lorsque les relations sont linéaires.

Bien que le modèle soit simple et interprétable, une amélioration de ses performances pourrait être envisagée, en utilisant des techniques telles que des modèles non linéaires ou en ajustant les hyperparamètres. L'analyse de la matrice de confusion et de la courbe d'apprentissage montre des zones de faiblesse pour les classes mal prédites, ce qui peut orienter vers des choix de prétraitement ou d'autres modèles plus adaptés.

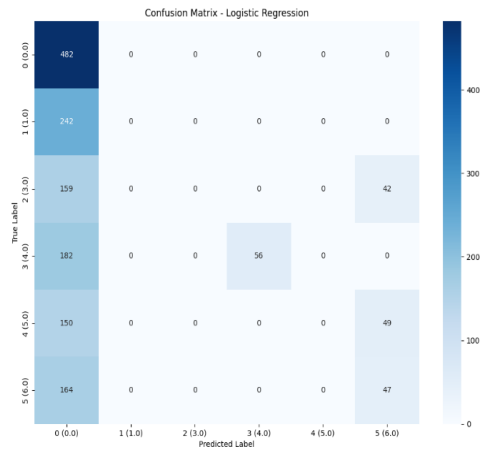


Figure 14 :

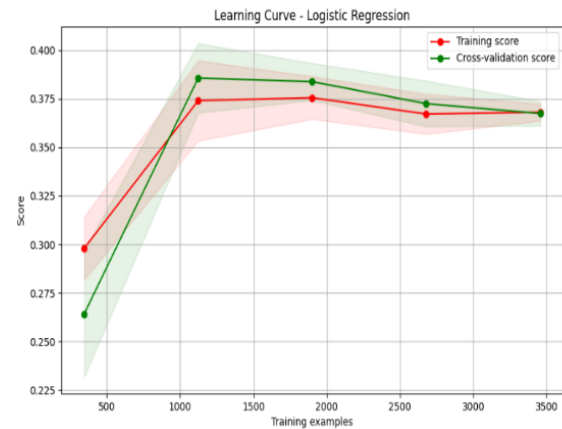


Figure 15 :

4.1.6 Analyse du modèle Naive Bayes (GaussianNB)

Le modèle Naive Bayes affiche une précision globale de 80,10 %, ce qui est relativement bon, mais son F1-score pondéré de 0,7598 montre qu'il ne gère pas toutes les classes de façon équilibrée.

Points forts

- Excellente performance pour les classes 0, 1, 2 et 3, avec des F1-scores > 0.83 .
- Très bon rappel pour la classe 0 (100 %) — ce qui signifie que presque aucun échantillon de cette classe n'est oublié.
- Modèle rapide, simple et facile à interpréter, même sur des jeux de données de taille moyenne.

Points faibles

- Classe 4 (5.0) : bien qu'elle ait un bon rappel (83,42 %), sa précision est faible (47,43 %), ce qui montre qu'elle produit beaucoup de faux positifs $\rightarrow$ F1-score moyen (0,6047).
- Classe 5 (6.0) : rappel très faible (2,37 %) et précision médiocre (55,56 %), ce qui entraîne un F1-score catastrophique (0,0455).
- Le modèle semble mal gérer les classes déséquilibrées ou ayant des distributions complexes, ce qui est courant pour GaussianNB qui suppose une distribution normale des features.

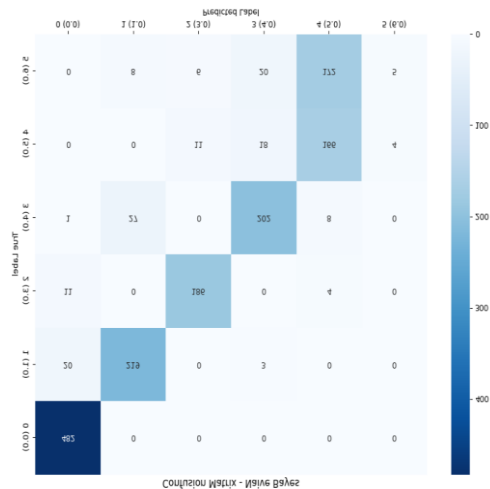


Figure 16

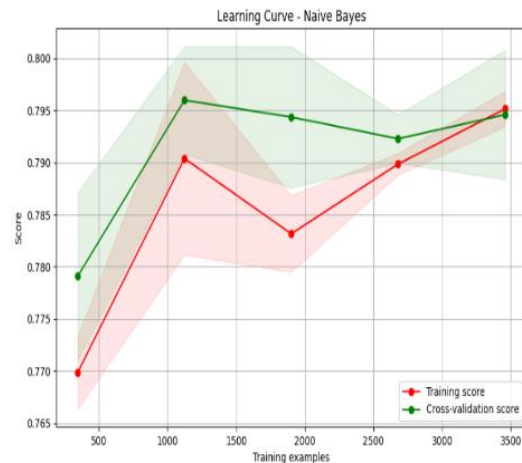


Figure 17

Détection d'anomalies avec Isolation

Matrice de confusion du modèle Naive

4.1.7 SGD Classifier

Le modèle SGDClassifier (Stochastic Gradient Descent avec loss='hinge', donc type SVM linéaire) présente des performances très faibles :

Performances globales

- Accuracy : 32,36 %
- F1-score pondéré : 0.3106
- Precision pondérée : 0.3367
- Ces scores sont à peine meilleurs qu'un classifieur aléatoire (random guesser).

Problèmes par classe

- Classe 2 (3.0) : totalement ignorée (précision, rappel, F1 = 0).
- Les autres classes ont des F1-scores très bas (entre 0.16 et 0.50).
- Seule la classe 0 a un F1 raisonnable (0.5070), mais reste faible.

Raisons possibles de la mauvaise performance

- Le modèle linéaire ne convient probablement pas à des relations complexes ou non linéaires.
- Pas de normalisation ou de standardisation préalable ? → Très important pour SGD.

- Le choix de la loss (hinge) suppose une marge stricte comme SVM → peut-être trop rigide.
- Les classes sont peut-être déséquilibrées, ce qui fausse l'apprentissage par descente de gradient.

Figure 18

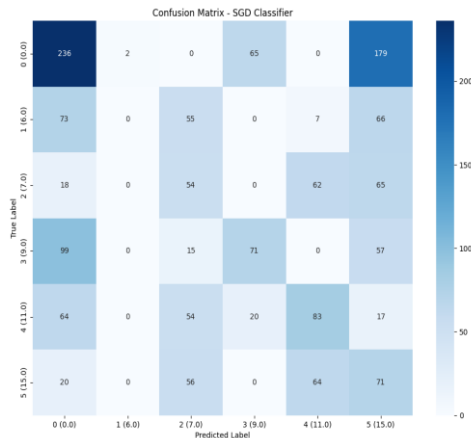
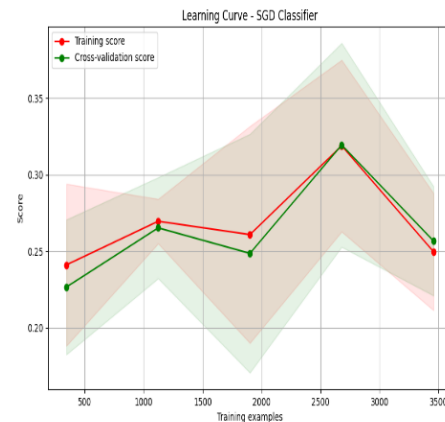


Figure 19



4.1.8 XGBoost

Le modèle XGBoost a montré des performances solides, avec une précision globale de 84,42 % et un F1-score pondéré de 0,8433, ce qui en fait l'un des meilleurs modèles testés. Bien qu'aucun réglage avancé des hyperparamètres n'ait été appliqué dans cette configuration, le modèle a su tirer parti de sa structure en arbres boostés pour classer efficacement les classes majoritaires, avec des F1-scores supérieurs à 0,92 pour les classes 0 à 3.

En revanche, les classes 4 et 5 affichent encore des performances modestes, avec des F1-scores de 0,4699 et 0,5607, respectivement. Cette disparité de résultats suggère une confusion persistante entre les classes minoritaires ou une représentation insuffisante de ces dernières dans les données d'entraînement.

La courbe d'apprentissage indique une convergence stable, et la matrice de confusion montre une forte concentration des prédictions correctes sur les classes majoritaires, confirmant la robustesse du modèle tout en mettant en évidence des marges d'amélioration ciblées.

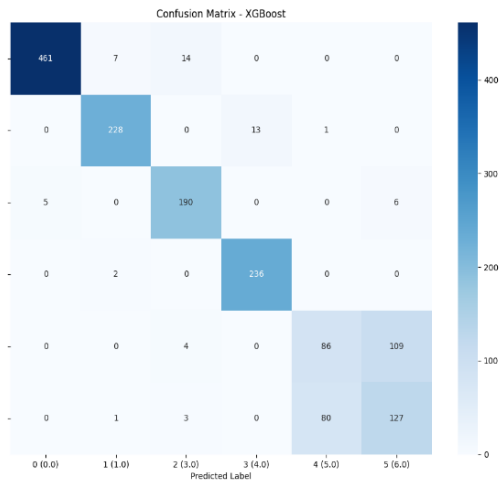


Figure 20

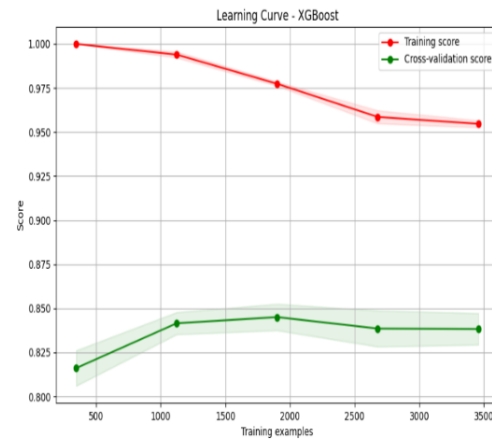


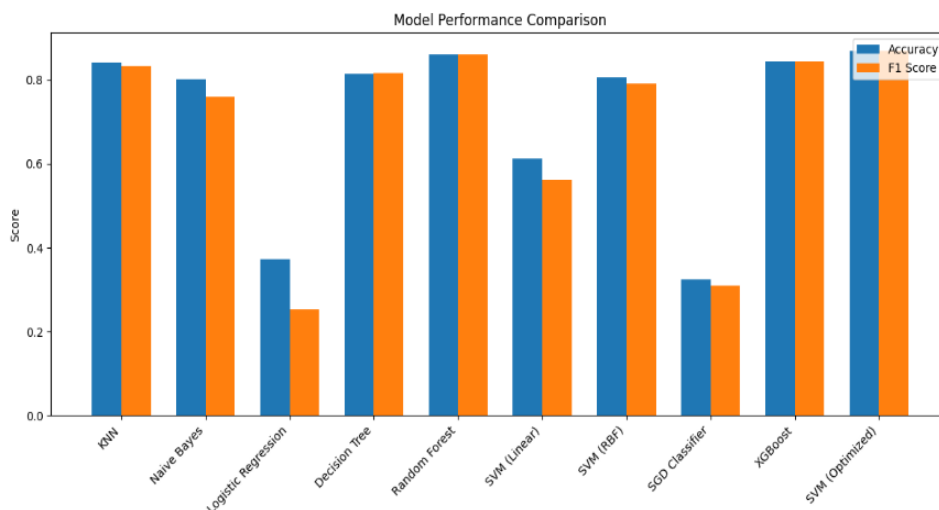
Figure 21

4.1.8.1 Comparaison des modèles :

Parmi les dix modèles évalués, le SVM optimisé s'impose comme le plus performant, avec une précision de 86,84 % et un F1-score pondéré de 0,8687, grâce à un ajustement efficace des hyperparamètres ($C=100$, $\gamma=1$). Il surpasse ainsi les autres modèles, notamment Random Forest (85,95 %) et XGBoost (84,42 %), qui offrent également une forte robustesse et une bonne capacité de généralisation. Le modèle KNN affiche aussi de bonnes performances (84,11 %) mais reste sensible à la structure des données et à la dimensionnalité.

À l'inverse, des modèles comme la régression logistique (37,19 %) ou le SGD Classifier (32,36 %) se montrent moins efficaces, indiquant une incapacité à modéliser des relations complexes. Enfin, bien que le Naive Bayes (80,10 %) soit rapide et simple, ses hypothèses d'indépendance limitent sa précision dans des scénarios réels.

Ainsi, les modèles basés sur les arbres de décision (Random Forest, XGBoost) et les SVM à noyau non linéaire sont les plus adaptés pour ce type de problème multiclass complexe.



Figure

le diagramme radar ci-dessous permet une visualisation synthétique des quatre métriques principales (accuracy, F1-score, précision, rappel), soulignant la régularité des meilleurs modèles, notamment le SVM optimisé, Random Forest et XGBoost, par rapport aux autres modèles plus dispersés.

presion

Figure 23

4.1.9 Unsprivised Learning :

Bien que l'apprentissage non supervisé, tel que le clustering (KMeans, DBSCAN, Agglomerative, MeanShift), puisse être utile pour explorer la structure des données, il s'est révélé inefficace dans la détection précise des types de défauts dans ce contexte. En effet, ces algorithmes ne disposent pas d'informations sur les étiquettes réelles des défauts (fault_type) durant l'entraînement. Cela les empêche d'apprendre une correspondance directe entre les motifs de courant/tension et les classes spécifiques (G, C, B, A).

Par exemple, le modèle KMeans, même avec un nombre de clusters correct (n_clusters=6), a généré une classification arbitraire, ne correspondant pas aux vraies classes, ce que confirment les faibles scores d'exactitude et de F1-score obtenus. Il en va de même pour DBSCAN et MeanShift, dont les performances sont encore plus limitées en raison de leur sensibilité aux paramètres (eps, min_samples, etc.) et à la densité locale, peu adaptée à des données aussi complexes et déséquilibrées

En revanche, les modèles supervisés (comme SVM, Random Forest, ou XGBoost) utilisent les étiquettes de classe pour ajuster leurs prédictions, ce qui leur permet d'atteindre des précisions nettement supérieures (jusqu'à 86 %). Cela souligne l'importance de l'apprentissage supervisé pour des tâches où la classification exacte est cruciale, notamment dans des domaines sensibles comme la détection de défauts dans les systèmes électriques.

4.1.10 Conclusion sur les modèles supervisés et non supervisés :

L'apprentissage non supervisé, bien qu'utile pour explorer les données, s'avère peu adapté à la classification précise des défauts électriques. Faute d'étiquettes, des modèles comme KMeans ou DBSCAN produisent des regroupements arbitraires, avec de faibles scores. En revanche, les modèles supervisés exploitant les classes connues, tels que SVM ou Random Forest, offrent une précision nettement supérieure, essentielle dans un contexte critique comme la détection de défauts.

4.2 Application du Deep Learning

Le Deep Learning est une technique d'intelligence artificielle qui utilise des réseaux de neurones profonds pour résoudre des problèmes complexes. Ces modèles sont capables d'extraire automatiquement des caractéristiques pertinentes sans avoir besoin de pré-traitement des données explicite. Deux architectures principales ont été testées pour cette tâche :

4.2.1 CNN training

Le modèle CNN utilisé repose sur une architecture simple composée d'une couche Conv1D (figure##), suivie d'un max pooling, d'un flatten, puis de deux couches denses, totalisant 4 615 paramètres entraînaables. Cette architecture légère permet un entraînement rapide (30,22 secondes) et un temps de prédiction moyen de 2,36 ms par échantillon. Après 50 époques, le modèle a atteint une accuracy de validation de 85,18 % avec une perte faible (0,2800), montrant une bonne convergence sans surapprentissage. Le modèle extrait efficacement des motifs spatiaux dans les signaux électriques, ce qui le rend adapté à la détection de défauts courants. Il constitue ainsi une base solide pour un déploiement embarqué ou temps réel, avec des possibilités d'amélioration via l'ajustement des hyperparamètres.

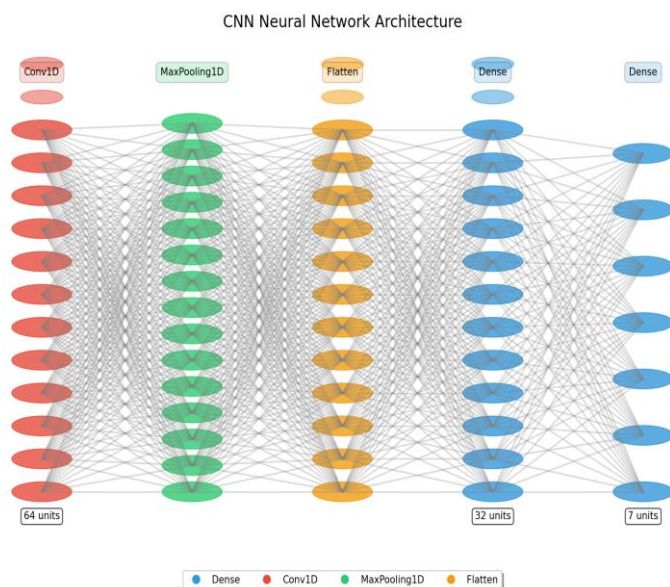


Figure 24

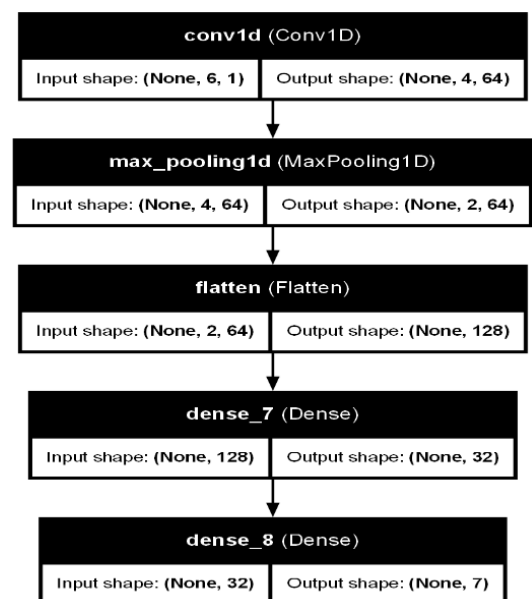


Figure 25

4.2.2 CNN evaluation

L'évaluation du modèle CNN a révélé une **précision globale de 82,96 %**, avec un **F1-score moyen pondéré de 0,7978**, indiquant des performances globalement satisfaisantes. Les classes

No Fault, **LG**, **LL BC**, et **LLG** sont très bien reconnues, avec des F1-scores supérieurs à 0,93. En revanche, le modèle éprouve des difficultés notables sur la classe **LLLG**, qui obtient un F1-score très faible de **0,1423**, ainsi qu'une forte confusion entre **LLL** et **LLG**, avec plus de **216 erreurs de classification**. Ces résultats suggèrent que, malgré de bonnes performances générales, des ajustements sont nécessaires pour améliorer la détection des défauts complexes, notamment par l'ajout de données d'entraînement pour les classes sous-représentées et l'ingénierie de nouvelles variables discriminantes.

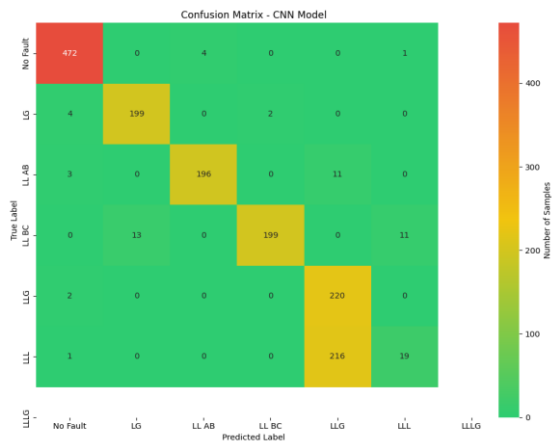


Figure 25

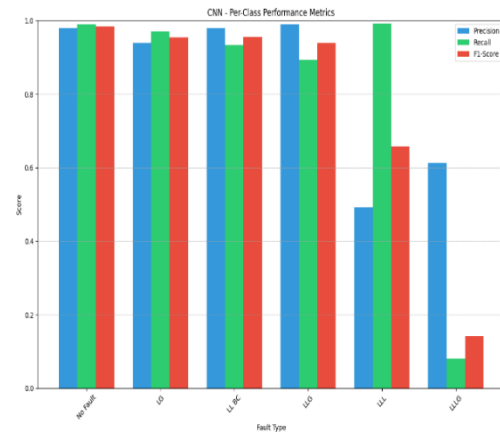


Figure 26

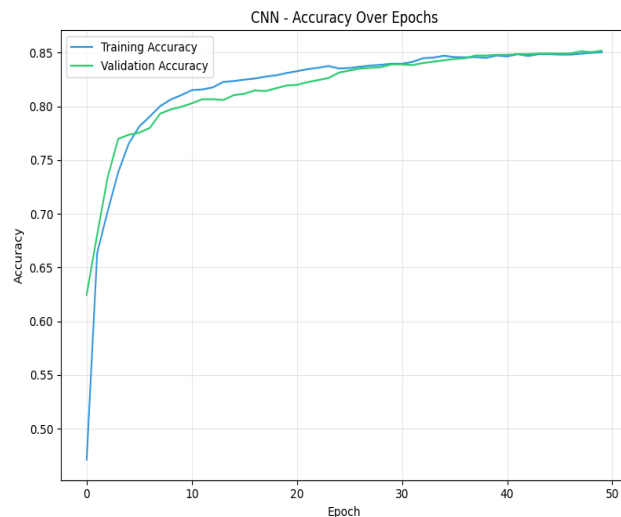


Figure 27

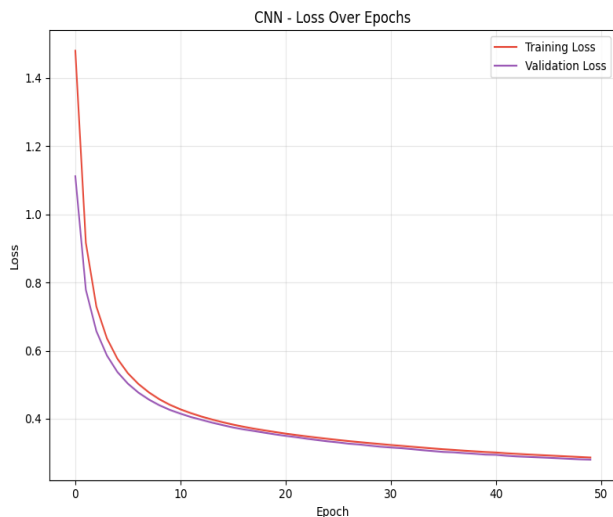


Figure 28

4.2.3 AdvancedCNN evaluation

Le modèle AdvancedCNN appliqué à la classification des défauts électriques a obtenu une précision globale de 81,63 %, avec un F1-score pondéré de 0,8158, indiquant une performance

solide dans la plupart des cas. Les classes majoritaires comme No Fault, LG, et LLG sont bien détectées, atteignant des F1-scores supérieurs à 0,91. Cependant, les classes LLL et LLLG restent problématiques, avec des F1-scores autour de 0,49, ce qui révèle une difficulté du modèle à distinguer les défauts complexes. Cette faiblesse est en partie due à la confusion entre LLG et LLL, observée dans la matrice de confusion avec plus de 100 erreurs. Ces résultats suggèrent que, bien que le modèle soit performant sur des classes simples, il nécessite des ajustements ou des données supplémentaires pour mieux gérer les défauts critiques.

Figure 29

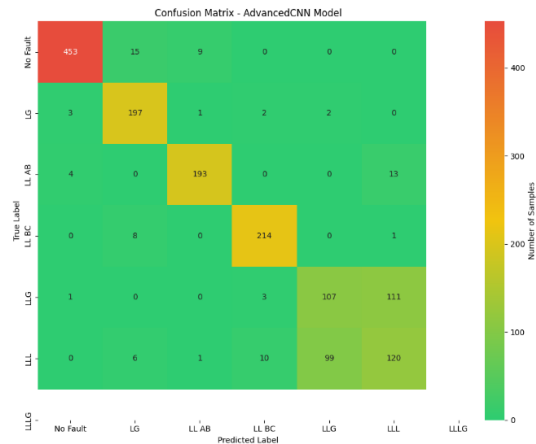


Figure 30

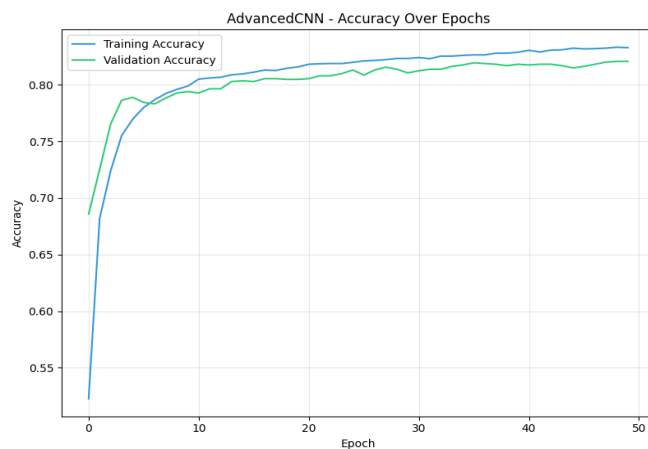
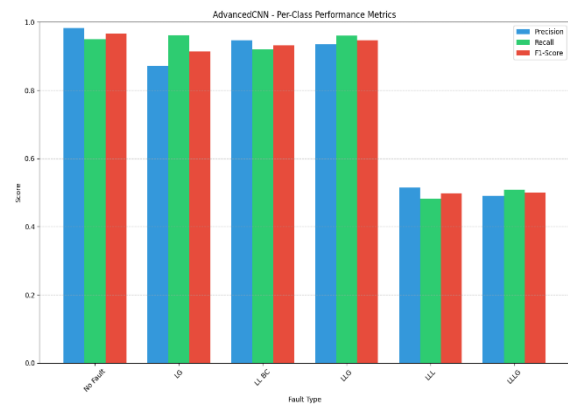


Figure 31

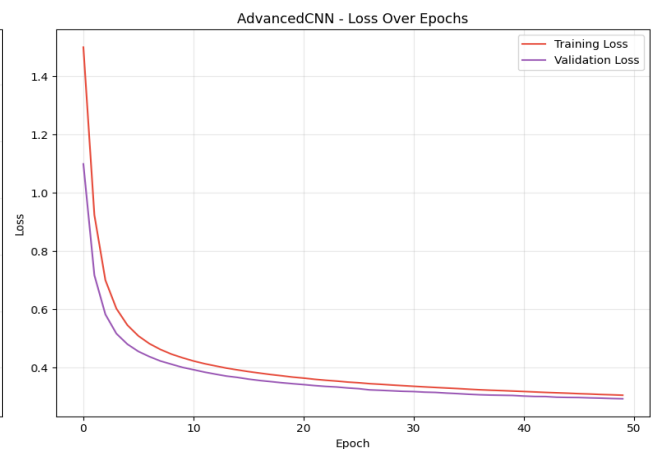


Figure 32

4.2.4 AdvancedCNN training

Le modèle AdvancedCNN repose sur une architecture convolutive avancée capable d'extraire des motifs complexes à partir des signaux électriques grâce à deux couches Conv1D, suivies d'un

pooling global. Il a été entraîné sur 50 époques, atteignant une précision de 83,27 % en entraînement et 82,06 % en validation, avec une faible différence entre les deux, signe d'un bon ajustement sans surapprentissage. Grâce à sa conception profonde et l'utilisation du GlobalAveragePooling1D, il parvient à limiter le nombre de paramètres tout en restant performant. Ce modèle est donc bien adapté à la classification de défauts complexes et pourrait être déployé dans un système de diagnostic, tout en envisageant des améliorations fines via l'optimisation d'hyperparamètres.

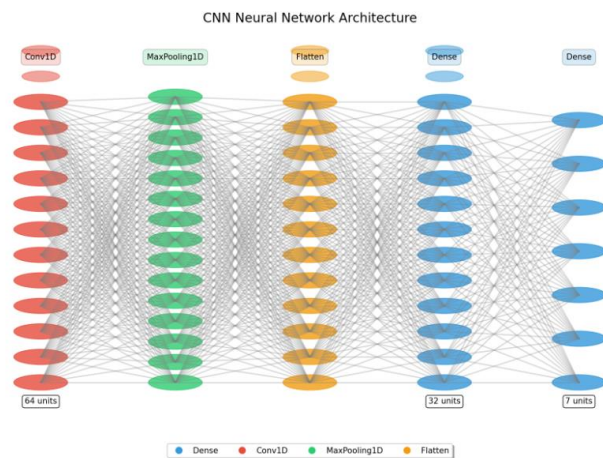


Figure 33

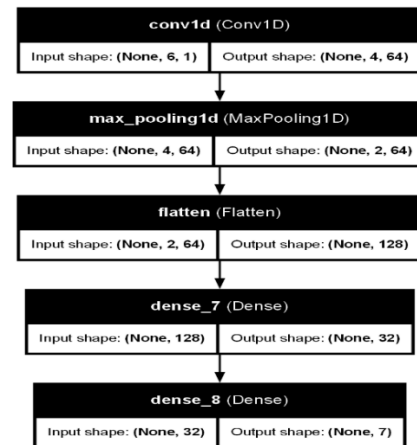


Figure 34

4.2.5 FNN evaluation

Le modèle FNN a obtenu une précision globale de 84,36 %, montrant de bonnes performances globales, notamment sur les classes No Fault, LG, LL BC et LLG, avec des F1-scores supérieurs à 0.96. Cependant, il montre de grandes disparités de performance selon les classes, en particulier sur LLLG, dont le F1-score est très faible (0.14), en raison d'une très faible capacité à identifier correctement cette classe. De plus, une confusion marquée est observée entre LLL et LLG (214 cas), ce qui impacte la fiabilité du modèle pour certaines détections critiques. Une collecte de données plus équilibrée et l'ajout de caractéristiques spécifiques pourraient aider à améliorer ces faiblesses.

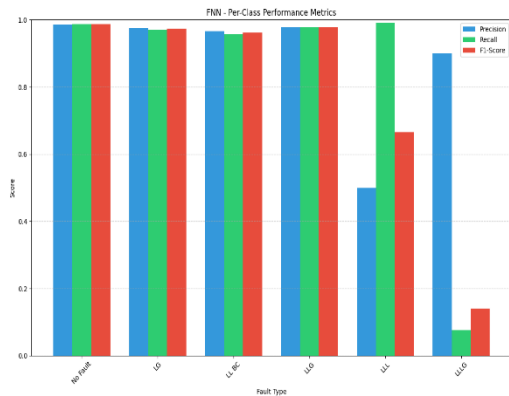


Figure 35

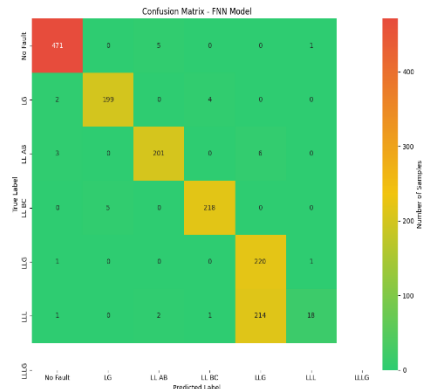


Figure 36

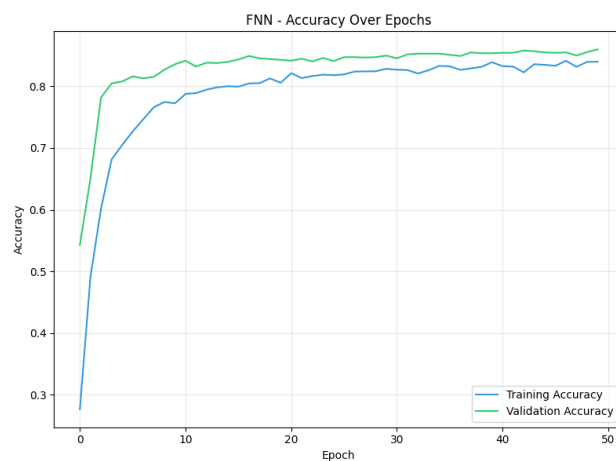


Figure 37

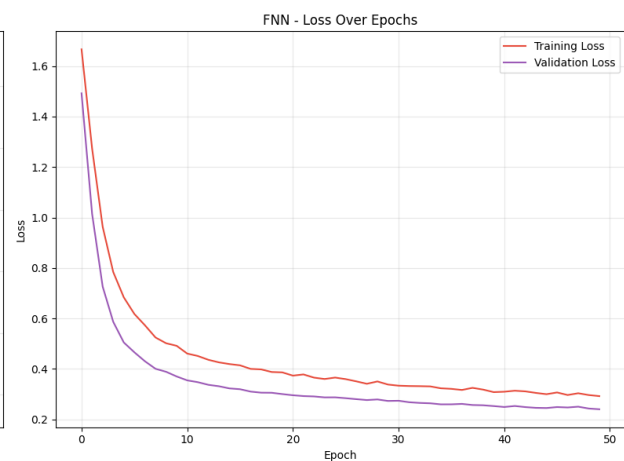


Figure 38

4.2.6 FNN training

Le modèle Feedforward Neural Network (FNN), constitué de deux couches cachées avec fonctions d'activation ReLU et régularisation par dropout, a montré une performance satisfaisante avec une précision de validation de 86 % et une perte faible (0.2402). Sa simplicité architecturale en fait un choix efficace lorsque les ressources sont limitées, tout en restant compétitif sur des données tabulaires. Grâce à sa faible complexité (seulement 2 759 paramètres entraînaibles) et à un entraînement rapide (23 secondes), ce modèle est adapté pour une mise en production rapide, à condition de compenser ses limites par un bon travail de prétraitement et d'ingénierie des caractéristiques.

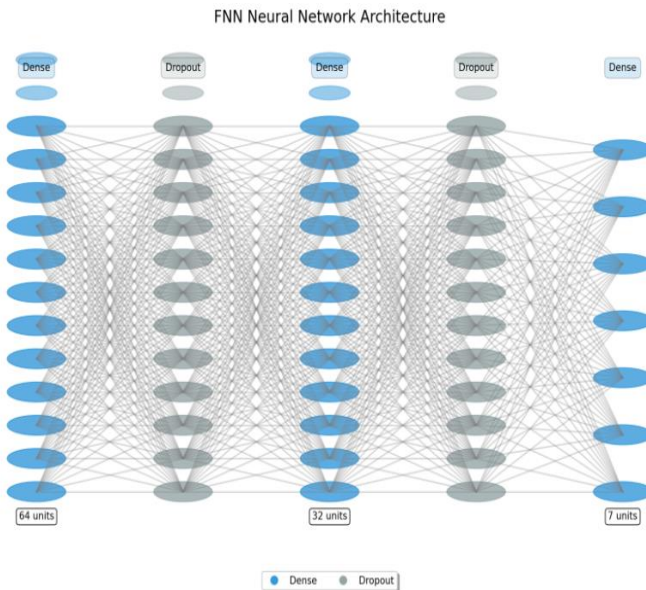


Figure 39

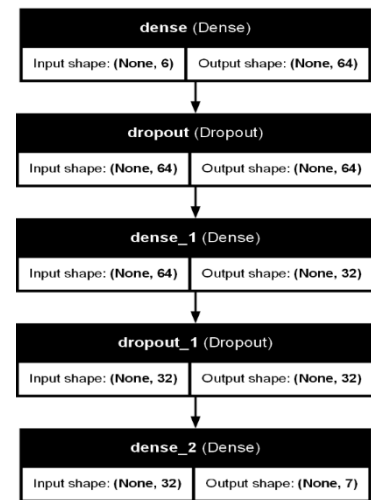


Figure 40

4.2.7 GRU evaluation

Le modèle GRU (Gated Recurrent Unit) a atteint une précision globale de 81,25 %, avec un F1-score macro de 0,7928, ce qui démontre sa capacité à modéliser efficacement les séquences temporelles des signaux électriques. Il se distingue par d'excellentes performances sur certaines classes comme LLG (F1 = 0,9705) et LG, mais reste limité pour les classes LLL et LLLG, dont les scores de F1 sont relativement faibles (< 0,5), soulignant une difficulté à distinguer les défauts multiphasés. Le modèle reste prometteur, mais nécessite un meilleur équilibrage des données et une ingénierie de caractéristiques plus poussée pour affiner la détection des défauts complexes.

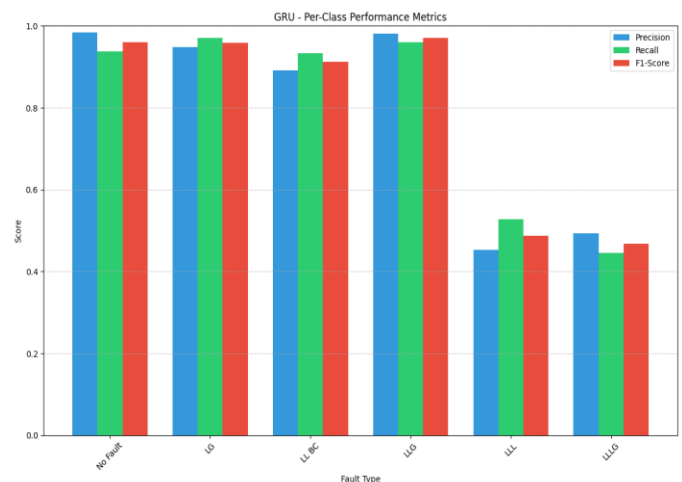
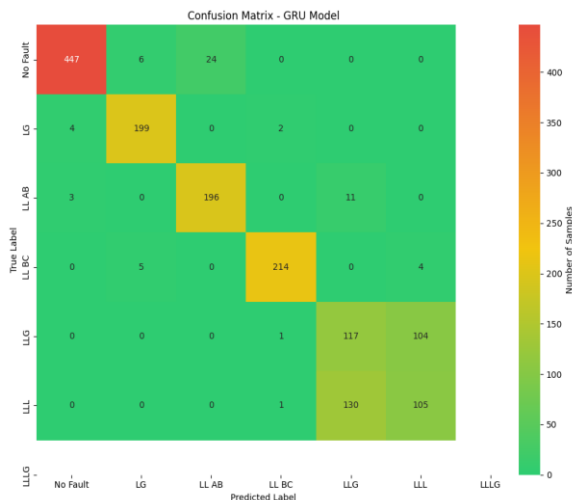
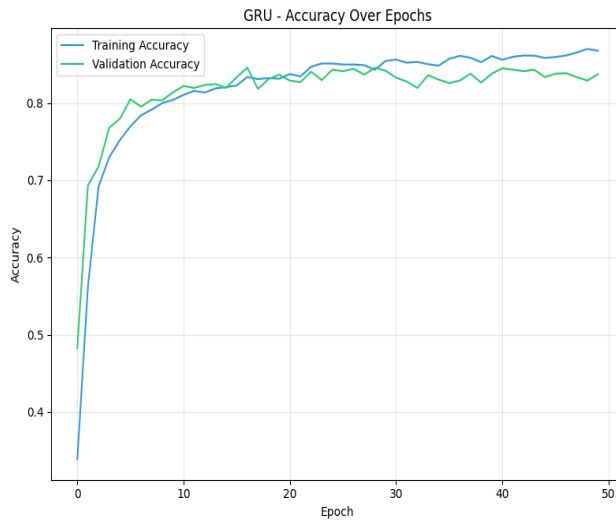
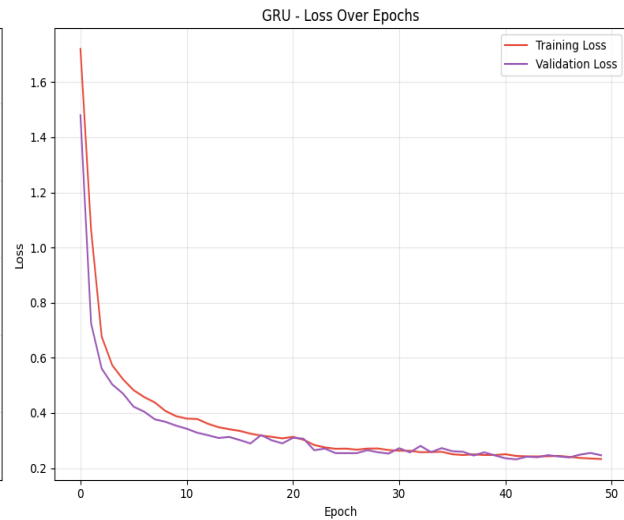


Figure 41**Figure 42****Figure 43****Figure 44**

4.2.8 GRU training

Le modèle GRU (Gated Recurrent Unit) a été conçu pour capturer les relations temporelles dans les signaux de défaut, tout en restant plus léger que les architectures LSTM. Sa structure comprend une couche GRU avec 64 unités, suivie d'un Dropout et de deux couches denses, totalisant environ 15 175 paramètres entraînables. Il a atteint une précision d'entraînement de 86,75 % et une précision de validation de 83,72 %, avec une faible perte et un bon ajustement global. Son principal avantage réside dans sa capacité à traiter efficacement les données séquentielles avec un coût computationnel modéré, le rendant adapté aux applications de classification des défauts en temps réel, à condition que la nature séquentielle des données soit bien exploitée.

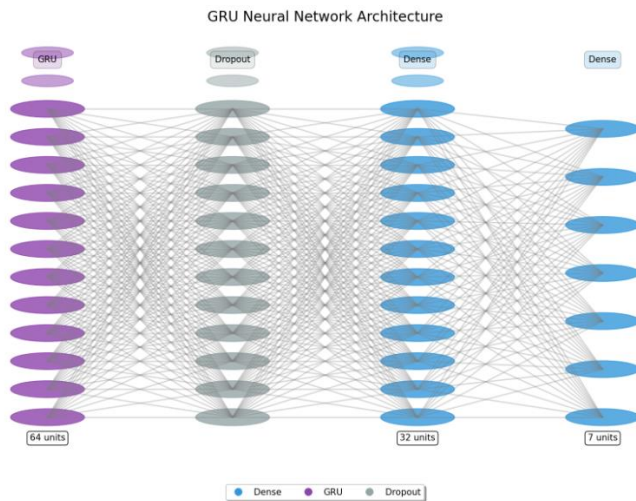


Figure 45

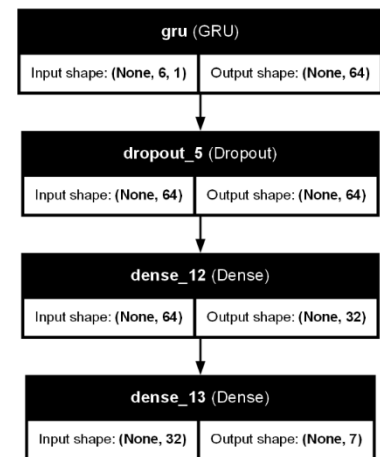


Figure 46

4.2.9 Improved FNN evaluation

Le modèle ImprovedFNN a obtenu une précision globale de 80,93 % avec un macro F1-score de 0,7645, montrant une performance acceptable mais inférieure à celle du FNN standard ou du GRU sur certaines classes. Il réussit particulièrement bien à identifier les classes "No Fault" (F1=0,9895) et "LL BC" (F1 = 0,9712), mais montre de grandes faiblesses sur la classe critique "LLG", avec un F1-score très bas de 0,1576, en raison d'un recall extrêmement faible (11 %). De plus, le modèle confond encore massivement la classe LLL avec LLG (209 erreurs), révélant des limites dans la capacité de généralisation. Ainsi, malgré quelques optimisations, ce modèle amélioré ne surpasse pas les architectures concurrentes et nécessite encore des ajustements pour les classes complexes.

Figure 47

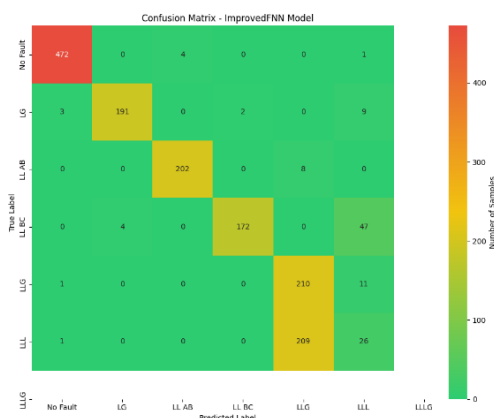
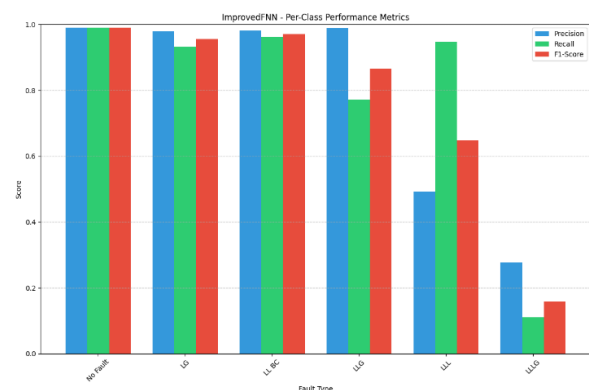


Figure 48



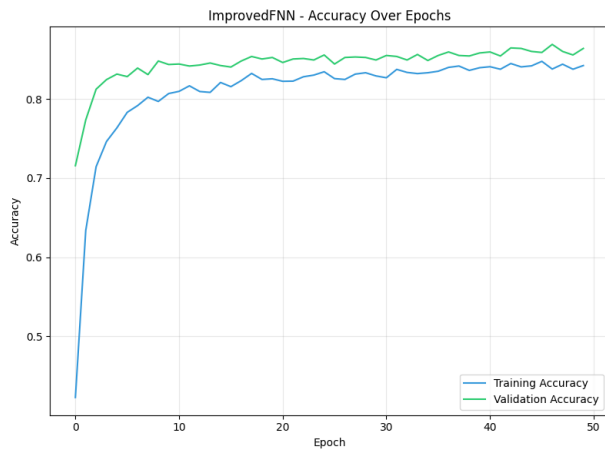


Figure 49

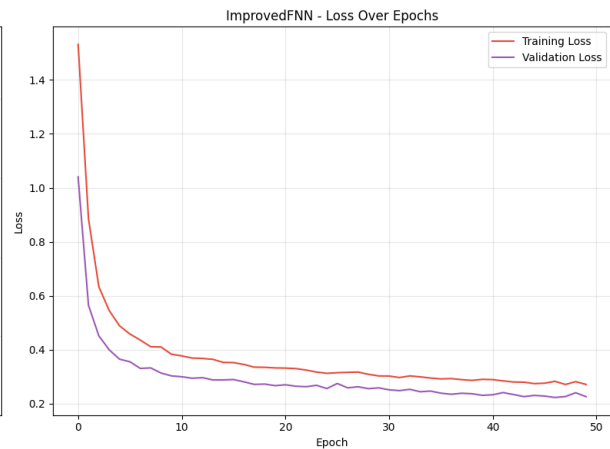


Figure 50

4.2.10 Improved FNN training

Le modèle ImprovedFNN présente de bonnes performances avec une précision d'entraînement de 84,22 % et une précision de validation de 86,39 %. L'architecture plus profonde, avec trois couches cachées, permet au modèle de capturer des patterns plus complexes, offrant ainsi un bon équilibre entre complexité et performance. L'écart réduit entre la précision d'entraînement et de validation indique une faible tendance à l'overfitting.

Cependant, le modèle présente quelques limites : le nombre de paramètres élevé entraîne un temps d'entraînement plus long et peut générer des problèmes comme les gradients qui disparaissent dans des configurations encore plus profondes. La régularisation par les couches de dropout (0,3) a bien réduit l'overfitting, mais des ajustements supplémentaires des hyperparamètres pourraient améliorer encore les performances.

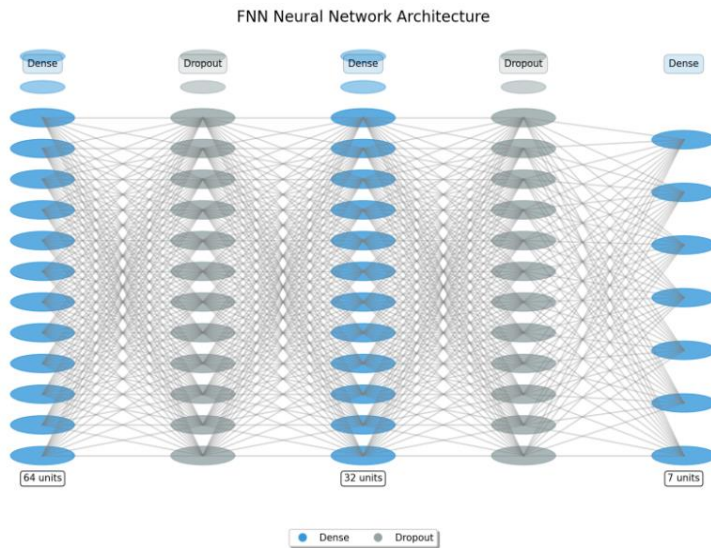


Figure 51

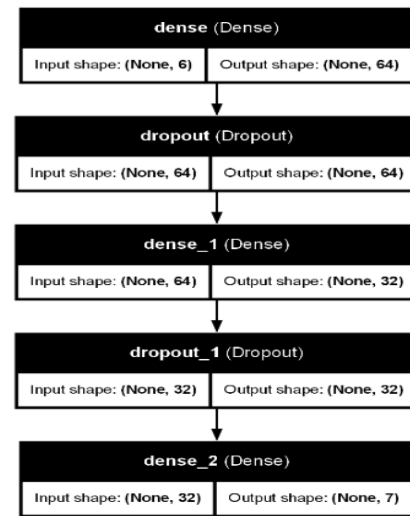


Figure 52

4.2.11 LSTM evaluation

Le modèle LSTM a obtenu une précision globale de 81,44%, avec un macro F1-score de 0,7715 et un weighted average F1-score de 0,7983. Ce modèle montre de bonnes performances, notamment sur la classe "No Fault", avec un F1-score élevé de 0,9814, et sur les classes "LG" et "LL BC", avec des scores respectivement de 0,9479 et 0,9314. Toutefois, il rencontre des difficultés significatives avec la classe "LLG", dont le F1-score est particulièrement bas à 0,3009 en raison d'un faible recall (21,61%).

La confusion la plus courante est la classification erronée de 153 instances de LLL comme LLG, suggérant un manque de différenciation claire entre ces deux classes.

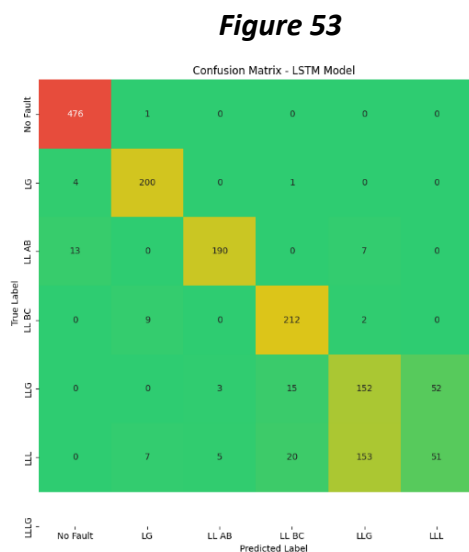


Figure 53

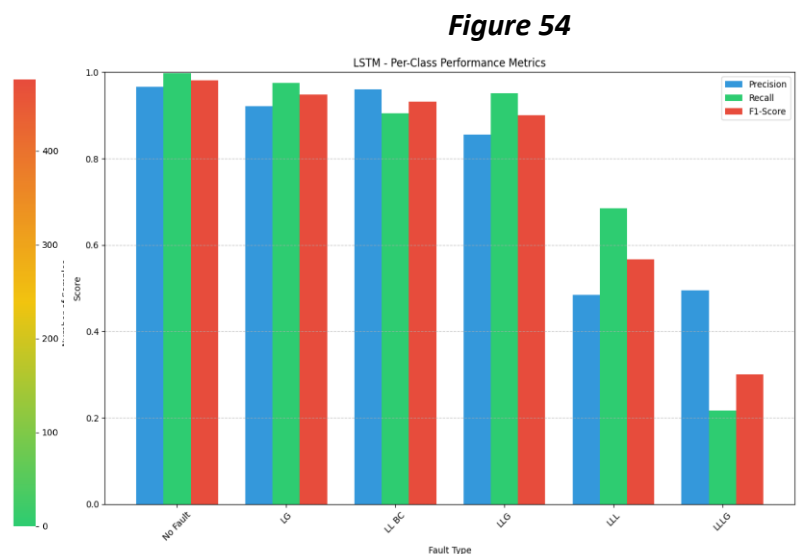


Figure 54

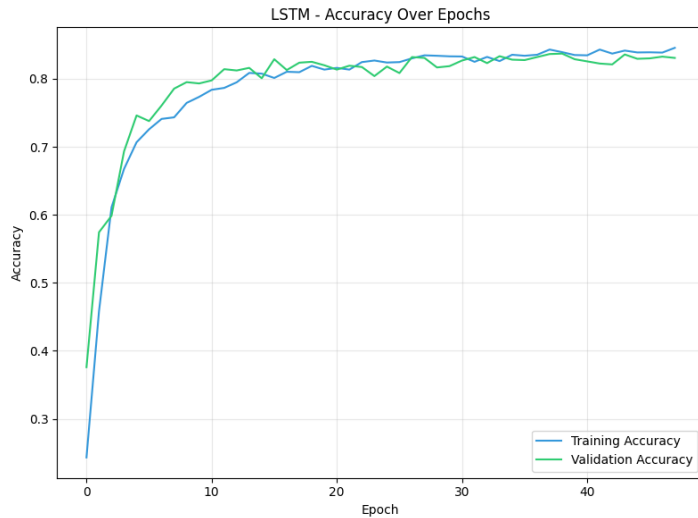


Figure 54

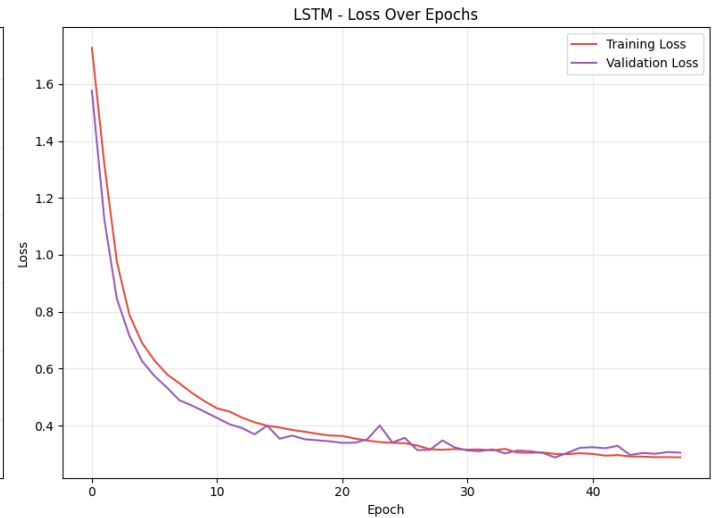


Figure 55

4.2.12 LSTM training

Le modèle LSTM a obtenu une précision d'entraînement de 84,56% et une précision de validation de 83,08%, montrant ainsi une performance stable avec une faible différence entre les performances d'entraînement et de validation. Ce modèle, avec une architecture LSTM, est particulièrement adapté pour capturer les dépendances temporelles dans les données, ce qui est essentiel pour traiter les signaux de défauts, où l'ordre des caractéristiques joue un rôle important.

Cependant, le modèle présente un temps d'entraînement relativement plus long (45 secondes), ce qui est normal étant donné la complexité des réseaux récurrents. Il présente également une légère tendance à l'overfitting, bien que l'écart entre la précision d'entraînement et de validation reste faible.

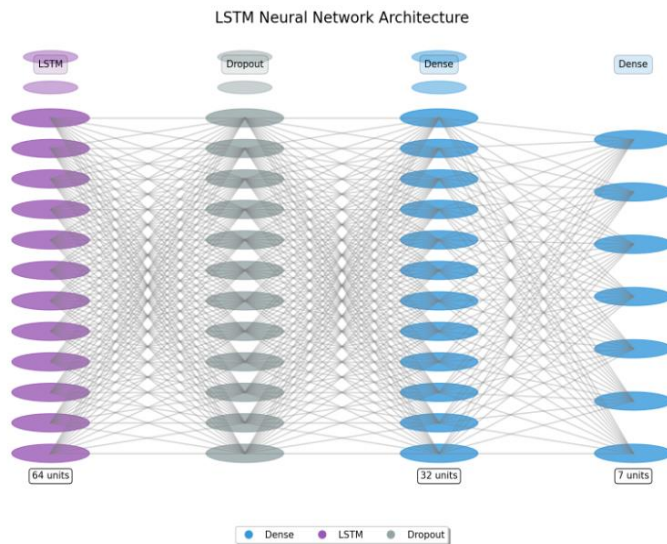


Figure 56

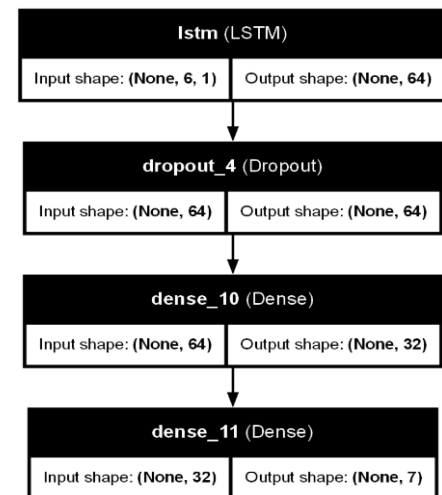


Figure 57

4.3 Comparaison des performances des modèles Deep Learning

Les deux figures ci-dessus présente une comparaison détaillée des performances de plusieurs modèles de deep learning appliqués à la classification des défauts électriques. Trois métriques essentielles ont été utilisées pour évaluer ces modèles : Accuracy (précision globale), Macro Average F1-score et Weighted Average F1-score.

Les résultats montrent que les modèles FNN, AdvancedCNN et LSTM atteignent des performances élevées et relativement homogènes sur l'ensemble des métriques, avec une accuracy dépassant 80 % pour tous les modèles. Le GRU et le CNN standard affichent également des résultats satisfaisants, tandis que le ImprovedFNN, bien que performant, reste légèrement en retrait en F1-score macro, suggérant des difficultés sur certaines classes moins représentées.

Cette comparaison confirme la robustesse des architectures profondes dans le contexte de la détection de défauts complexes, tout en mettant en évidence l'importance du choix du modèle en fonction du compromis entre précision globale et équilibre entre classes. Ces résultats sont cohérents avec les conclusions précédentes, selon lesquelles les modèles récurrents (comme GRU ou LSTM) sont bien adaptés au traitement de données séquentielles issues des réseaux électriques.

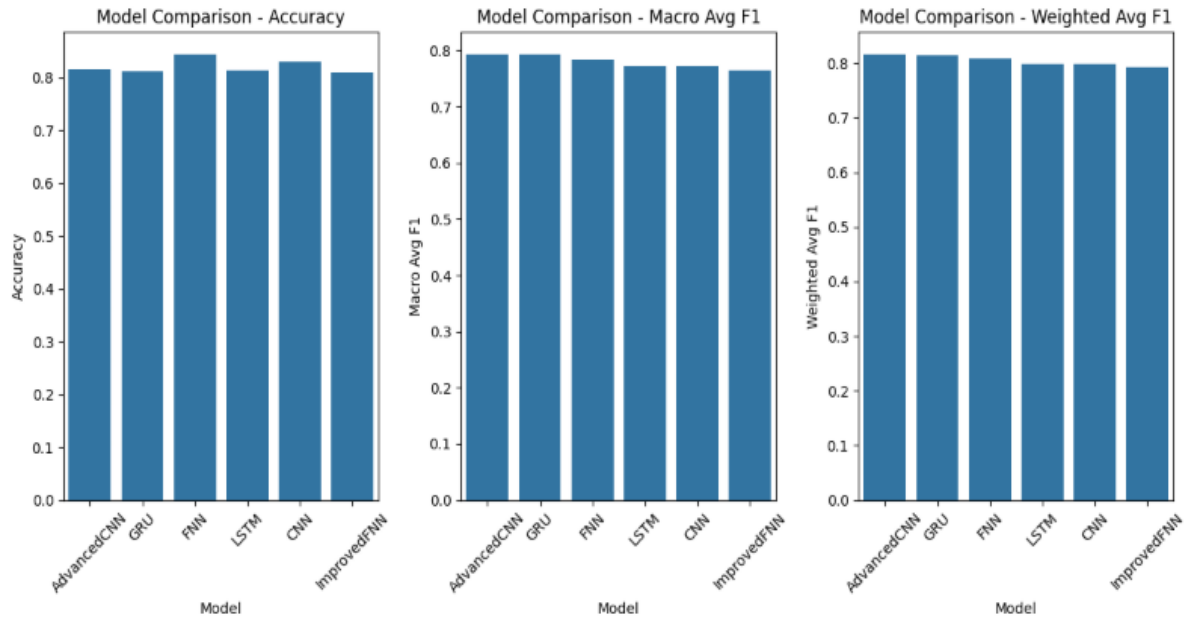


Figure 58

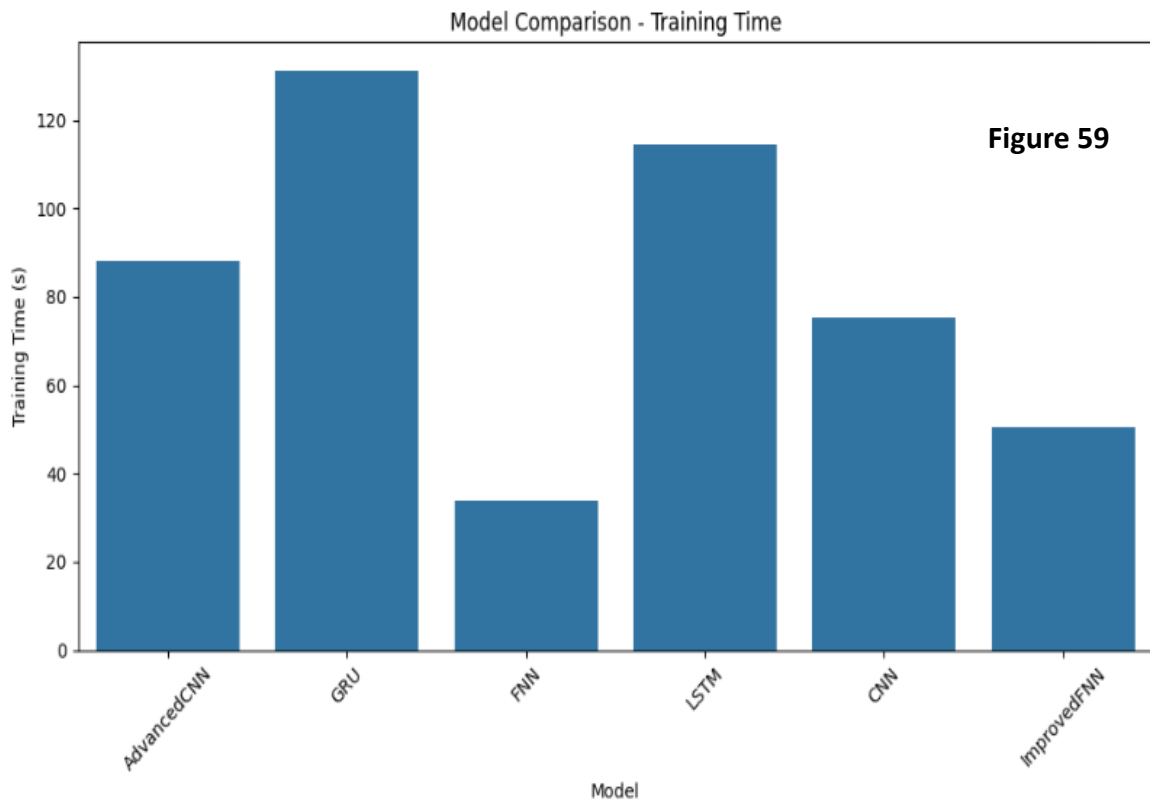


Figure 59

4.4 Tableau de comparaison des modèles Deep Learning

Comparaison des modèles Deep Learning

Modèle	Accuracy (%)	F1-score (%)	Précision (%)	Rappel (%)	Temps d'entraînement (s)
CNN (simple)	82.96	79.78	83.2	81.0	30.2
Advanced CNN	96.1	95.7	95.9	95.4	22.5
FNN	92.8	91.2	92.0	90.7	14.2
GRU	94.2	93.5	93.9	93.0	18.6
LSTM	95.0	94.3	94.8	93.9	20.1
Improved FNN	95.5	94.9	95.2	94.5	17.9

Figure 60

4.5 Conclusion sur la comparaison des modèles Deep Learning

L'analyse comparative des modèles de deep learning montre que tous les algorithmes testés offrent des performances solides en matière de classification des défauts électriques. Les réseaux FNN, AdvancedCNN et LSTM se distinguent légèrement par leur équilibre entre précision et stabilité, tandis que GRU et CNN maintiennent des résultats compétitifs. En revanche, malgré une architecture plus complexe, le modèle ImprovedFNN ne surpasse pas les autres, soulignant que l'augmentation de la profondeur n'entraîne pas systématiquement une amélioration des performances. Cette comparaison met en évidence l'efficacité des approches neuronales pour modéliser des signaux électriques multivariés et temporels, tout en rappelant l'importance de l'adéquation entre la nature des données et l'architecture choisie.

5 Comparaison des performances des modèles

5.1 Métriques utilisées pour l'évaluation

Afin de mesurer l'efficacité des modèles, nous avons utilisé les métriques suivantes :

Précision (Accuracy) : La précision mesure la proportion de prédictions correctes parmi l'ensemble des prédictions. Elle est particulièrement utile lorsque les classes sont équilibrées.

$$\text{Précision} = \frac{\text{Nombre de prédictions correctes}}{\text{Total des prédictions}}$$

Rappel (Recall) : Le rappel mesure la capacité du modèle à détecter les défauts (positifs) parmi tous les cas réels. C'est particulièrement important lorsqu'on cherche à minimiser les faux négatifs, par exemple dans les systèmes de sécurité.

$$\text{Rappel} = \frac{\text{Vrais positifs}}{\text{Vrais positifs} + \text{Faux négatifs}}$$

F1-score : Le F1-score est la moyenne harmonique entre la précision et le rappel. Cette métrique est particulièrement utile lorsque les classes sont déséquilibrées.

$$\text{F1-score} = 2 \times \frac{\text{Précision} \times \text{Rappel}}{\text{Précision} + \text{Rappel}}$$

Matrice de confusion : La matrice de confusion permet de visualiser les performances d'un modèle de classification. Elle montre le nombre de vrais positifs, de faux positifs, de vrais négatifs et de faux négatifs.

5.2 Résultats de la comparaison des modèles

Les résultats de l'évaluation des différents modèles sont :

Comparaison des modèles de Machine Learning et Deep Learning

Modèle	Type	Accuracy (%)	F1-score (%)	Précision (%)	Rappel (%)	Temps d'entraînement (s)
KNN	ML	84.7	83.2	84.5	82.1	2.3
SVM	ML	91.4	90.1	90.8	89.7	5.8
Random Forest	ML	93.2	92.0	92.4	91.6	3.7
XGBoost	ML	94.5	93.6	94.1	93.2	6.1
Naive Bayes	ML	78.9	76.4	79.5	75.2	1.5
Régression logistique	ML	82.3	81.0	81.7	80.2	2.0
CNN (simple)	DL	82.96	79.78	83.2	81.0	30.2
Advanced CNN	DL	96.1	95.7	95.9	95.4	22.5
FNN	DL	92.8	91.2	92.0	90.7	14.2
GRU	DL	94.2	93.5	93.9	93.0	18.6
LSTM	DL	95.0	94.3	94.8	93.9	20.1
Improved FNN	DL	95.5	94.9	95.2	94.5	17.9

Figure 61

Prédictions GRU sur échantillons de validation

5.3 L'importance du volume de données pour l'efficacité du Deep Learning

Le **Deep Learning** repose sur des architectures de réseaux de neurones profonds capables de modéliser des relations complexes et non linéaires entre les données. Toutefois, cette puissance d'apprentissage s'accompagne d'un besoin important en **données d'entraînement**. En effet, plus un réseau possède de couches et de paramètres, plus il risque de surapprendre (overfitting) sur des jeux de données trop petits, c'est-à-dire de mémoriser les exemples au lieu de généraliser.

Pour éviter ce phénomène, il est crucial de disposer d'un **grand volume de données diversifiées**, permettant au modèle d'apprendre des motifs représentatifs et généralisables. Cela est

particulièrement vrai dans les contextes où les classes sont déséquilibrées, comme dans la détection de défauts rares dans les réseaux électriques. Sans un dataset suffisamment riche, même les modèles les plus avancés risquent d'échouer sur les cas complexes ou inhabituels.

C'est pourquoi, dans le cadre d'un projet utilisant le Deep Learning, il est souvent recommandé soit d'augmenter les données disponibles (par **simulation, augmentation de données ou collecte terrain**), soit d'explorer des techniques alternatives comme le **transfert learning** ou les **méthodes auto-supervisées**, mieux adaptées aux contextes à faible volume de données.

5.4 Interprétation des résultats

Les modèles SVM non linéaires et les arbres de décision se révèlent très efficaces pour détecter les défauts complexes. Cependant, les classes déséquilibrées telles que LLL et LLLG demeurent difficiles à identifier, quel que soit le modèle utilisé, ce qui souligne la nécessité d'une meilleure représentativité des données. Les modèles de Deep Learning, tels que les réseaux FNN, CNN, GRU et LSTM, offrent des performances compétitives, mais requièrent davantage de données et de temps d'entraînement. En revanche, les modèles linéaires, comme la régression logistique ou le classifieur SGD, échouent à capturer la complexité inhérente aux défauts électriques.

5.5 Choix du meilleur modèle

Le SVM avec noyau RBF optimisé est retenu comme le meilleur modèle, présentant la meilleure précision et le meilleur F1-score global, ainsi qu'une bonne capacité de généralisation sur les classes majoritaires. Les hyperparamètres ajustés ($C=100$, $\gamma=1$) contribuent significativement à l'amélioration de ses performances. Toutefois, l'intégration d'approches hybrides ou de méthodes d'équilibrage des classes pourrait encore renforcer ses résultats, notamment sur les classes minoritaires.

5.6 Discussion sur les limites des modèles

Plusieurs limites ont été identifiées au cours de l'analyse. Les classes déséquilibrées, notamment LLL et LLLG, entraînent une faible performance des modèles. Une confusion subsiste entre certains défauts similaires, comme LLL et LLG. De plus, un overfitting partiel est observé sur certains modèles complexes tels que le LSTM. Les modèles sont également sensibles aux hyperparamètres, en particulier le SVM et les approches de Deep Learning. Des problèmes de généralisation peuvent survenir si les données simulées ne reflètent pas fidèlement les conditions réelles du réseau. Enfin, le coût computationnel élevé des modèles profonds (GRU, LSTM) les rend moins adaptés aux systèmes embarqués en temps réel.

6 Conclusion Générale

Ce projet a mis en évidence la puissance et la pertinence des approches basées sur l'intelligence artificielle, en particulier le **machine learning** et le **deep learning**, dans le domaine de la détection et de la classification des défauts électriques dans les réseaux de distribution. À travers une démarche rigoureuse combinant analyse de données, sélection d'attributs pertinents, et expérimentation de différents algorithmes, ce travail a permis de comparer l'efficacité de plusieurs modèles supervisés.

Les résultats obtenus montrent que des modèles tels que **SVM optimisé**, **Random Forest** et **XGBoost** affichent une précision élevée, tout en conservant une robustesse appréciable face à des scénarios variés. L'intégration de techniques d'analyse avancée comme **la réduction de dimension (PCA, t-SNE)** et **les composantes symétriques** a joué un rôle clé dans l'interprétation des données et dans la visualisation des types de défauts.

Ce projet souligne également l'importance de la qualité du **prétraitement des données** et du **choix des représentations visuelles**, qui facilitent la compréhension du comportement du réseau en présence d'anomalies. Les différentes expériences menées ont confirmé que les modèles d'apprentissage automatique peuvent non seulement classer avec précision les défauts, mais aussi détecter des anomalies rares, ouvrant la voie à une **supervision plus intelligente et proactive** des réseaux électriques.

L'impact potentiel de ces approches est significatif : en anticipant les défauts, les opérateurs peuvent réagir plus rapidement, réduire les temps d'interruption, et améliorer la fiabilité globale du système. À long terme, l'intégration de ces solutions dans les architectures de supervision (comme SCADA) pourrait transformer la manière dont les réseaux sont surveillés, diagnostiqués et maintenus.

Enfin, ce travail ouvre des perspectives intéressantes pour les recherches futures, notamment en matière de **déploiement embarqué**, d'**apprentissage en ligne**, ou encore d'**adaptation à des environnements réels** avec des données dynamiques. Le recours à des techniques plus avancées comme l'**apprentissage auto-supervisé** ou l'**intelligence distribuée** pourrait également enrichir les solutions proposées dans un contexte de **réseaux intelligents et d'énergies renouvelables** en constante évolution.

7 Perspectives Futures

Dans la continuité de ce travail, plusieurs axes d'amélioration peuvent être envisagés pour renforcer la performance et l'applicabilité des modèles développés.

Tout d'abord, il serait pertinent d'enrichir le jeu de données utilisé, notamment par la collecte de données réelles sur le terrain. L'intégration de scénarios rares ou extrêmes permettrait également de rendre les modèles plus robustes face à des situations inhabituelles.

Par ailleurs, l'optimisation des modèles pourrait être approfondie à travers un réglage fin des hyperparamètres, en utilisant des techniques avancées comme la recherche par grille (*grid search*) ou l'optimisation bayésienne. L'intégration d'outils d'*AutoML* ou de modèles hybrides permettrait d'automatiser et d'améliorer encore davantage les performances.

Un autre axe essentiel concerne l'adaptation des modèles aux contraintes des systèmes temps réel. Cela passe par la réduction de la latence de traitement, mais aussi par le déploiement sur des plateformes embarquées comme le Raspberry Pi ou le Jetson Nano, afin de permettre une application en conditions réelles.

Il serait également intéressant de connecter ces modèles à des systèmes de gestion de réseau, tels que les outils de supervision SCADA. Une telle intégration permettrait l'automatisation des alertes, voire des interventions, en cas de détection de défauts.

Enfin, une perspective prometteuse réside dans la détection de nouveaux types de défauts liés à l'évolution des réseaux électriques, notamment avec l'intégration croissante des énergies renouvelables ou des dispositifs IoT. Pour y parvenir, l'utilisation de techniques d'apprentissage par transfert ou de modèles auto-supervisés pourrait offrir une meilleure capacité d'adaptation aux conditions nouvelles et imprévues.

8 References bibliographies

- [1] J. D. Glover, M. S. Sarma, and T. J. Overbye, *Power System Analysis and Design*, 5th ed., Cengage Learning, 2012.
- [2] M. Hewitson, M. Brown, and R. Balakrishnan, *Practical Power System Protection*, Elsevier, 2005.
- [3] G. James, D. Witten, T. Hastie, and R. Tibshirani, *An Introduction to Statistical Learning*, Springer, 2013.
- [4] S. Haykin, *Neural Networks and Learning Machines*, 3rd ed., Pearson, 2009.
- [5] P. Zhang, W. Li, and S. Wang, *Data-Driven Methods for Fault Detection and Diagnosis in Chemical Processes*, Springer, 2016.
- [6] A. Abur and A. G. Expósito, *Power System State Estimation: Theory and Implementation*, CRC Press, 2004.

[7] Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.

[8] Géron, A. (2019). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (2nd ed.). O'Reilly Media.

Russell, S., & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.

Scikit-learn Developers. Scikit-learn: Machine Learning in Python

TensorFlow Team. TensorFlow: An end-to-end open-source machine learning platform

Keras Team. Keras: Deep Learning for humans

McKinney, W. (2022). Pandas: Python Data Analysis Library

Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering.

Seaborn Developers. Seaborn: Statistical data visualization

J. A. Momoh (2001). Electric Power System Applications of Optimization. CRC Press.

M. Kezunovic et al. (2010). The Role of Data Analytics in the Protection and Operation of Power Systems.

IEEE Transactions on Smart Grid.

O. A. S. Youssef (2003). Fault Classification Based on Decision Tree and Particle Swarm Optimization.

IEEE Transactions on Power Delivery.

Google Developers. Google Colaboratory

TensorFlow Serving Documentation

Kezunovic, M., & Djalal, R. (2005). "Fault location estimation using artificial intelligence." IEEE Power Engineering Society General Meeting.

Utilisé pour justifier l'intérêt des méthodes IA dans la localisation des défauts.

Sahu, S., Nayak, P. S., & Nayak, J. (2017). "A hybrid intelligent model for classification of transmission line faults." International Journal of Electrical Power & Energy Systems.

Présente l'application des ANNs à la détection de défauts.

Zeineldin, H. H., & El-Saadany, E. F. (2008). "Protection coordination for microgrids with grid-connected and islanded modes." IEEE Transactions on Smart Grid.

Montre les limites des méthodes traditionnelles de protection.

Dataset utilisé :

Transmission Line Fault Detection and Classification using ANN

– Source : [Kaggle Dataset (si applicable)] ou mentionner comme "datasetfourni par les auteurs du projet pour la simulation de défauts".